

Sales Force -Last Mile Admin Project

Project Name: SkyCast – Smart City Weather Information System

Problem Statement: Modern cities are increasingly affected unpredictable weather events such as heavy rainfall, heatwaves, air pollution, and storms. Citizens, city administrators, and emergency services often lack real-time, localized weather insights to make quick, data-driven decisions.

Current systems are either generic weather apps (that do not provide city-specific microclimate data) or disjointed dashboards with no integration into smart city infrastructure.

SkyCast aims to bridge this gap by building a **Smart City Weather Information System** that delivers hyperlocal weather forecasts, air quality data, and emergency alerts to citizens, businesses, and government agencies.

Phase 1: Problem Understanding & Industry Analysis

1. Requirement Gathering

- Collect **functional requirements**:
 - Real-time weather updates (temperature, rainfall, wind speed, humidity)
 - Air quality index monitoring
 - Smart city integration (IoT sensors, APIs from meteorological departments)
 - User-specific notifications (alerts for storms, floods, heatwaves)
- Collect **non-functional requirements**:
 - High availability and scalability
 - Data security and compliance (e.g., for IoT sensor data)
 - Multi-platform support (web, mobile, smart kiosks)

2. Stakeholder Analysis

- **Citizens** → Need accurate, real-time weather alerts for daily planning & safety
- **City Administrators** → Require dashboards for urban planning, traffic management, disaster preparedness
- **Emergency Services** → Need instant alerts for floods, storms, fire hazards, etc.
- **Businesses** (logistics, transport, retail, agriculture) → Rely on weather insights for operations
- **Meteorological Agencies** → Provide raw data & forecasts
- **Citizens & Businesses** – Need accurate, real-time weather and air quality information to plan daily activities, travel, and operations safely.

3. Business Process Mapping

AS-IS Process (Current Scenario):

- Citizens rely on generic weather apps → limited accuracy
- Administrators depend on scattered reports from different agencies
- No central platform for **real-time insights & alerts**

TO-BE Process (With SkyCast):

1. IoT weather sensors + external APIs collect live data
2. Data processed and stored in Salesforce/Cloud platform
3. SkyCast dashboard provides **real-time weather visualization**
4. Automated **alerts/notifications** sent to citizens & stakeholders
5. Integration with **smart city infrastructure** (traffic lights, public transport updates, emergency services)

4. Industry-specific Use Case Analysis

- **E-Governance (Smart Cities in India, Singapore, Dubai):** Real-time pollution monitoring dashboards
- **Transportation & Logistics:** Companies like Uber & DHL use weather insights to optimize routes
- **Agriculture:** Hyperlocal forecasts help farmers plan irrigation & crop protection
- **Telecom & Energy:** Companies adjust networks and grids based on weather impact

5. AppExchange Exploration

- **Salesforce AppExchange** already has apps for:
 - Weather Forecast APIs integration
 - IoT data connectors
 - Emergency management & notifications
- Gaps found:
 - No **city-level hyperlocal solution** combining weather, AQI, and alerts
- Decision:
 - Use available **weather data connectors** from AppExchange
 - Build **custom SkyCast dashboards, alerts, and integrations** for smart cities

Phase 2: Org Setup & Configuration

1. Salesforce Editions

Choosing the right Salesforce Edition is a crucial step in configuring the SkyCast – Smart City Weather Information System. Each edition provides different levels of scalability, customization, and integration capabilities. After detailed evaluation, the Enterprise Edition was selected as the most suitable option.

- Evaluation of Editions: Professional, Enterprise, and Unlimited editions were compared for scalability, workflow automation, and API support.
- Selection Criteria: Considered factors like number of users, API limits, integration requirements, and cost feasibility.
- Final Choice: Enterprise Edition chosen for its balance of advanced automation and cost efficiency.
- Developer Org Usage: Developer Edition used for testing, training, and proof of concept activities.
- License Allocation: Licenses mapped for Admins, Standard Users, and API users.
- Justification: Enterprise Edition ensures scalability for future growth of the SkyCast system.

2. Company Profile Setup

The company profile setup establishes the foundational configuration for the Salesforce org. This ensures accurate localization, compliance, and brand identity within the system.

- Organization Details: Configured company name, address, and primary contact information.
- Locale & Time Zone: **Set to Indian Standard Time (IST) and English (India).**
- Default Currency: Enabled INR as the base currency and multi-currency support for USD/EUR.
- Branding: Uploaded company logo and configured email settings for notifications.
- Audit & Compliance: Verified organization information accuracy to prevent data/reporting issues.
- Governance: Ensured profile setup aligns with corporate policies.

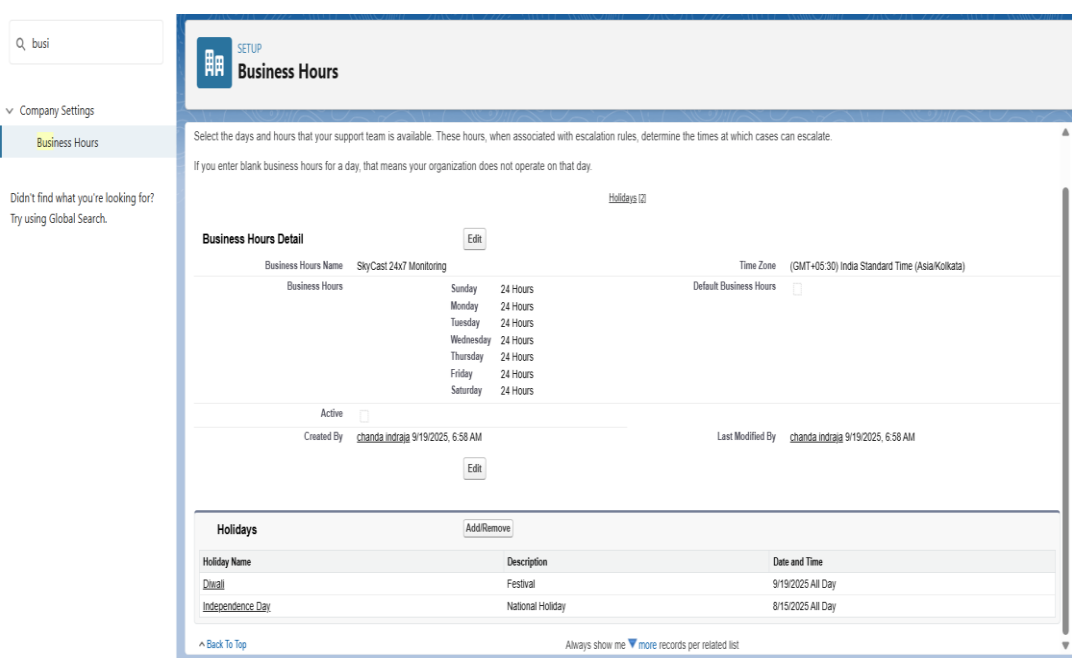
The screenshot displays the Salesforce Setup interface. The left sidebar shows the 'Setup' menu with 'Company Information' selected. The main content area is titled 'Company Information' and shows the organization's profile for 'ksrm college of engineering'. The profile includes details such as Organization Name, Primary Contact, Division, Address, Fiscal Year, and various settings like Newsletter, Admin Newsletter, and Locale Formats. The page also shows links for User Licenses, Permission Set Licenses, Feature Licenses, and Usage-based Entitlements. The bottom of the page indicates the profile was created by 'OrgFarm EPIC' on 7/4/2025 at 1:20 AM and modified by 'OrgFarm EPIC' on 7/13/2025 at 8:38 PM.

Organization Detail	
Organization Name	ksrm college of engineering
Primary Contact	OrgFarm EPIC
Division	
Address	United States
Fiscal Year Starts In	January
Activate Multiple Currencies	☐
Enable Data Translation	☐
Newsletter	☒
Admin Newsletter	☒
Hide Notices About System Maintenance	☐
Hide Notices About System Downtime	☐
Locale Formats	ICU
Phone	
Fax	
Default Locale	English (United States)
Default Language	English
Default Time Zone	(GMT-07:00) Pacific Daylight Time (America/Los_Angeles)
Currency Locale	English (United States) - USD
Used Data Space	350 KB (7%) View
Used File Space	17 KB (0%) View
API Requests, Last 24 Hours	0 (15,000 max)
Streaming API Events, Last 24 Hours	0 (10,000 max)
Restricted Logins, Current Month	0 (0 max)
Salesforce.com Organization ID	00DGL000006e9p8
Organization Edition	Developer Edition
Instance	CAN98

3. Business Hours & Holidays

Business hours and holiday settings define availability for service-level agreements, escalation rules, and customer interactions. Proper configuration ensures accurate case handling and timely response to emergencies.

- **Standard Hours:** Defined 9 AM – 6 PM, Monday to Friday.
- **Holiday Calendar:** Configured key national holidays such as Republic Day, Independence Day, and Diwali.
- **Emergency Hours:** Created 24/7 service hours for City Administrators and Emergency Services.
- **Escalation Rules:** Linked escalation and case management with business hours and holidays.
- **Testing:** Verified that escalation pauses during defined holidays.
- **Regional Variations:** Added holidays for state-specific requirements.



4. Fiscal Year Settings

Feature	Configuration / Action	Impact / Notes
Standard Fiscal Year	April–March cycle (aligned with Indian financial year)	Ensures all reports and forecasts follow standard FY
Custom Fiscal Year	Evaluated but deferred	Maintains simplicity and avoids unnecessary complexity
Revenue & Forecast Alignment	Revenue, Opportunities, and ARR tracking aligned to fiscal year	Accurate revenue recognition and forecasting
Reporting & Analytics	Dashboards and reports configured with fiscal year filters	Consistent reporting and analytics across the org
Governance	Restricted changes to fiscal year	Prevents accidental changes; ensures long-term consistency

5. User Setup & Licenses

a) User Types & Roles

- **Admin Users:** Full system-level permissions for configuration and management.
- **Standard Users:** Role-based access tailored to responsibilities.
- **API Users:** Dedicated for IoT sensor and system integrations.

b) License Management

- Assigned based on job function to ensure compliance and cost-efficiency.
- Optimized allocation to prevent under- or over-licensing.

c) Security Measures

- **Multi-Factor Authentication (MFA):** Enforced for all privileged accounts.
- **Login IP Ranges:** Restricted to trusted locations to reduce security risks.

The screenshot displays the Salesforce 'Setup' interface, specifically the 'Users' section. The left sidebar shows the navigation menu with 'Users' selected. The main content area is divided into two sections: 'All Users' and 'User Detail'.

All Users Section:

On this page you can create, view, and manage users.
To get more licenses, use the Your Account app. [Let's Go](#)

View: **All Users** | [Edit](#) | [Create New View](#)

Navigation: A B C D E F G H I J K L M N O P Q R S T U V W X Y Z Other **All**

Action	Full Name	Alias	Username	Role	Active	Profile
☐ Edit	Admin, SkyCast	sadmi	skycastadmin@skycast.com	Marketing	✓	Standard Platform User
☐ Edit	Chatter, Expert	chatter	chatter.00000000000000000000000000000000@chatter.salesforce.com		✓	Chatter Free User
☐ Edit	EPIC, OrgFarm	CEPIC	epic.711d980d3952@orgfarm.salesforce.com		✓	System Administrator
☐ Edit	Indira, Chandra	cha	chandandraya119@agentforce.com		✓	System Administrator
☐ Edit	Mikaelson, Kol	nmika	kol.mikaelson@2635.com	Inventory	✓	Platform 1
☐ Edit	Mikaelson, Niklaus	nmika	niklaus.mikaelson@123.com	Sales	✓	Platform 1
☐ Edit	User, Integration	integ	integration@00000000000000000000000000000000.com		✓	Analytics Cloud Integration User
☐ Edit	User, Security	sec	insightsecurity@00000000000000000000000000000000.com		✓	Analytics Cloud Security User

Navigation: A B C D E F G H I J K L M N O P Q R S T U V W X Y Z Other **All**

Buttons: [New User](#) [Reset Password\(s\)](#) [Add Multiple Users](#)

User Detail Section:

User: **SkyCast Admin** | [User Profile Help for this Page](#)

[Permission Set Assignments \(0\)](#) | [Permission Set Assignments Activation Required \(0\)](#) | [Permission Set Group Assignments \(0\)](#) | [Permission Set License Assignments \(0\)](#) | [Personal Groups \(0\)](#) | [Public Group Membership \(0\)](#) | [Queue Membership \(0\)](#) | [Team \(0\)](#) | [Managers in the Role Hierarchy \(0\)](#) | [OAuth Apps \(0\)](#) | [Third-Party Account Links \(0\)](#) | [Built-In Authentications \(0\)](#) | [Installed Mobile Apps \(0\)](#) | [Authentication Settings for External Systems \(0\)](#) | [Login History \(0\)](#) | [User Provisioning Accounts \(0\)](#)

User Detail | [Edit](#) | [Sharing](#) | [Reset Password](#) | [Freeze](#) | [View Summary](#)

Name	SkyCast Admin	Role	Marketing
Alias	sadmi	User License	Salesforce Platform
Email	chandandraya@gmail.com (Verify)	Profile	Standard Platform User
Username	skycastadmin@skycast.com	Active	✓
Nickname	User17562909362374596406	Marketing User	☐
Title		Offline User	☐
Company		Knowledge User	☐
Department		Flow User	☐
Division		Service Cloud User	☐
Address		Site.com Contributor User	☐
Time Zone	(GMT+05:30) India Standard Time (Asia/Colombo)	Site.com Publisher User	☐
Locale	English (United States)	WOC User	☐
Language	English	Mobile Push Registrations	View
Delegated Approver		Data.com User Type	A
Manager		Accessibility Mode (Classic Only)	☐ A
Receive Approval Request Emails	Only if I am an approver	Debug Mode	☐ A
Federation ID		High Contrast Palette on Charts	☐ A

6. Profiles

- **Profile Details**

- **Name:** Einstein Agent User
- **User License:** Einstein Agent
- Includes quick links to set **Login IP Ranges**, **Enabled Apex Class Access**, **External Data Source Access**, and other permissions.

- **Page Layout Assignments**

- **Global Layout** – Default global page layout
- **Home Page Default** – Home page configuration
- **Account Layout** – Page layout for Account object
- Additional layouts for Alternative Payment Method, Appointment Invitation, Location Group, Macro, Object Milestone, Operating Hours, and Opportunity.

- **Customization Options**

- You can **Edit**, **Clone**, or **View Users** directly from the profile page.
- Field-level security and tab visibility can be adjusted here to show only the relevant applications per profile.

The screenshot shows the Salesforce Setup interface for the 'Einstein Agent User' profile. The left sidebar contains navigation links for Setup, Home, and Object Manager. The main content area is titled 'Profiles' and shows the profile details for 'Einstein Agent User'. The profile is associated with the 'Einstein Agent' user license. The 'Page Layouts' section lists various layouts assigned to the profile, including Global Layout, Home Page Default, Account Layout, Alternative Payment Method Layout, Appointment Invitation Layout, Location Group Layout, Location Group Assignment Layout, Macro Layout, Object Milestone Layout, Operating Hours Layout, and Opportunity Layout. The 'Profile Detail' section includes buttons for 'Edit', 'Clone', and 'View Users'.

Setup
Profiles

Profile: Einstein Agent User

Users with this profile have the permissions and page layouts listed below. Administrators can change a user's profile by editing that user's personal information.

If your organization uses Record Types, use the Edit links in the Record Type Settings section below to make one or more record types available to users with this profile.

Login IP Ranges [0] | Enabled Apex Class Access [0] | Enabled External Data Source Access [0] | Enabled Named Credential Access [0] | Enabled External Credential Principal Access [0] | Enabled Custom Metadata Type Access [0] | Enabled Custom Setting Definitions Access [0] | Enabled Flow Access [0] | Enabled Service Presence Status Access [0] | Enabled Custom Permissions [0]

Profile Detail [Edit] [Clone] [View Users]

Name	Einstein Agent User
User License	Einstein Agent
Created By	salesforce.com, inc., 7/4/2025, 1:20 AM
Modified By	chanda.indraja, 7/15/2025, 1:04 AM

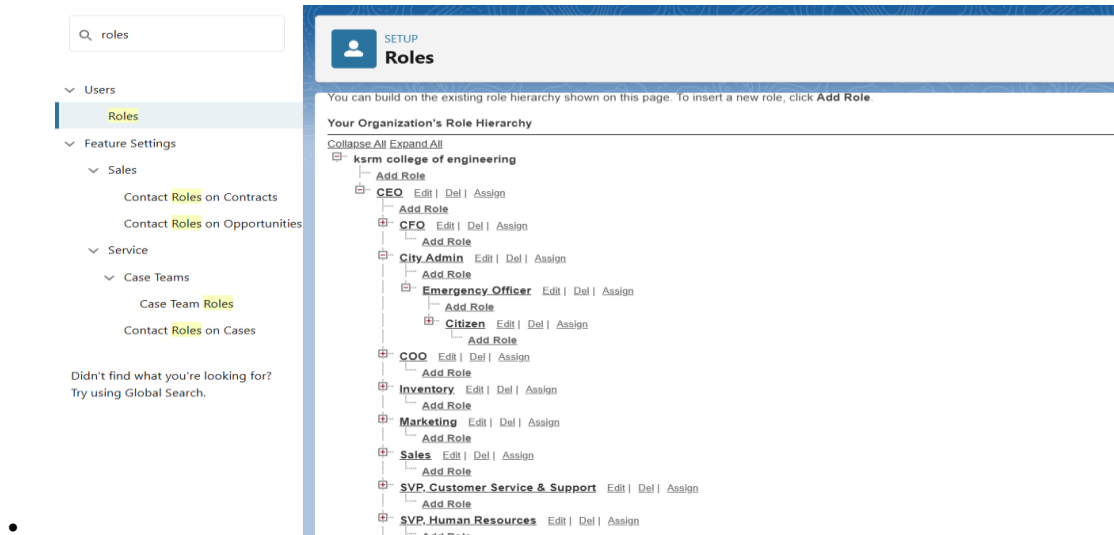
Page Layouts

Standard Object Layouts	Global	Global Layout [View Assignment]	Location Group	Location Group Layout [View Assignment]
Email Application	Not Assigned	[View Assignment]	Location Group Assignment	Location Group Assignment Layout [View Assignment]
Home Page Layout	Home Page Default	[View Assignment]	Macro	Macro Layout [View Assignment]
Account	Account Layout	[View Assignment]	Object Milestone	Object Milestone Layout [View Assignment]
Alternative Payment Method	Alternative Payment Method Layout	[View Assignment]	Operating Hours	Operating Hours Layout [View Assignment]
Appointment Invitation	Appointment Invitation Layout	[View Assignment]	Opportunity	Opportunity Layout [View Assignment]

7. Roles

Roles control data visibility and hierarchical reporting structures in Salesforce. In SkyCast, they reflect the responsibilities of different smart city stakeholders.

- **City Administrator:** Top-level role with access to all records.
- **Emergency Services:** Role created with access to alerts and cases.
- **Business Partners:** Role with access to weather analytics and insights.
- **Citizens:** Limited access through community/portal roles.



8. Permission Sets

Objective

Provide selected users with **read-only access** to the custom object *Weather Data* without changing their base profile permissions.

Background

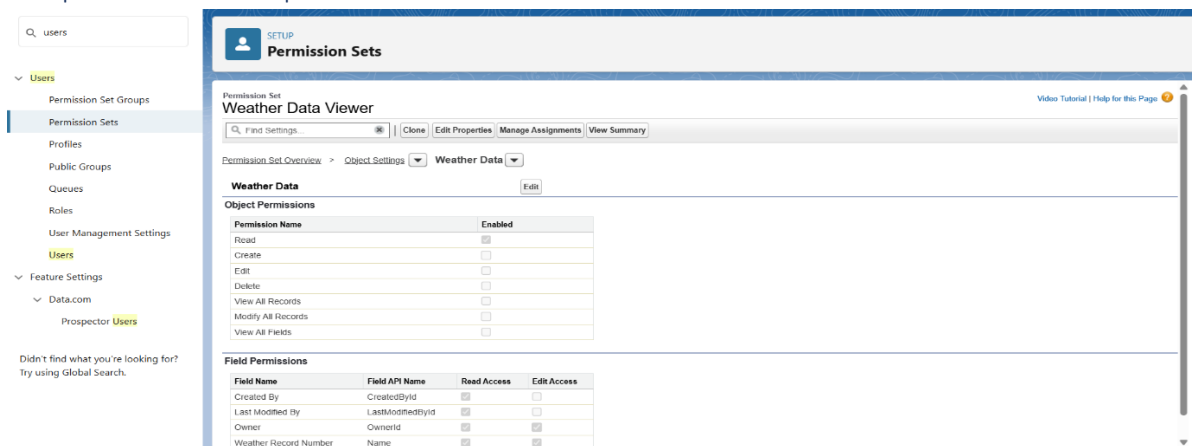
In Salesforce, **Permission Sets** extend a user's access beyond what their profile grants. They are ideal when:

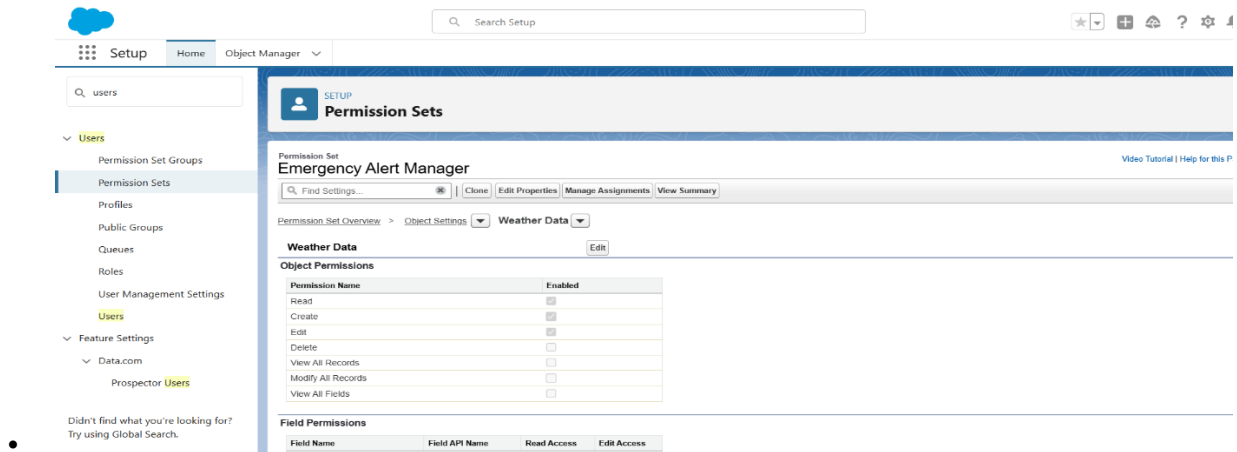
- Only certain users need extra permissions.
- You don't want to clone or modify profiles.

Configuration Details

Item	Description
Permission Set Name	Weather Data Viewer
Assigned Object	Weather Data (custom object)
Object Permissions	Read access only (Create, Edit, Delete disabled)
Field Permissions	Read access for all key fields: <i>Created By</i> , <i>LastModifiedBy</i> , <i>Owner</i> , <i>Weather Record Number</i> , <i>Name</i>
Assignments	Users who require visibility into weather records without edit rights

Implementation Steps





9. Organization-Wide Defaults (OWD)

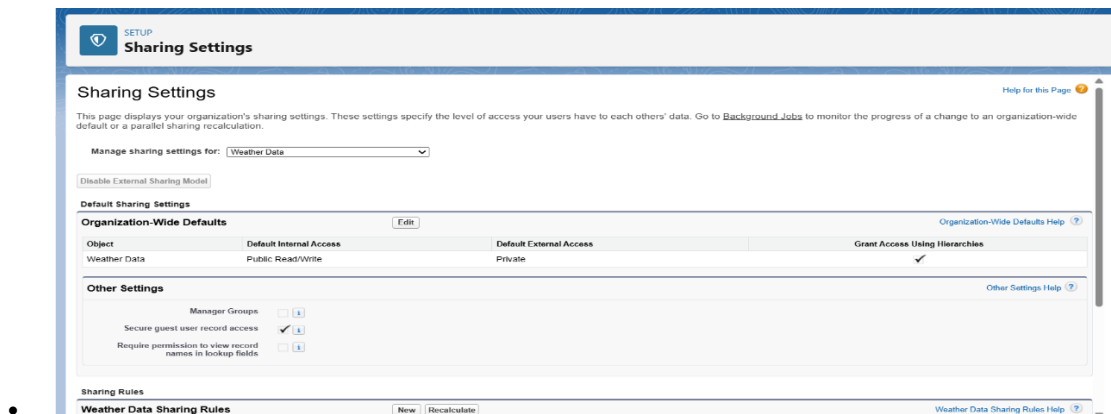
OWD settings define the baseline level of data access across the org. SkyCast required a balance between transparency of weather data and privacy for sensitive information.

- Accounts: Set to Private for confidentiality.
- Weather Data: Configured as Public Read-Only to allow transparency.
- Emergency Alerts: Access controlled via parent relationships.
- Cases: Configured with private visibility to protect sensitive reports.
- Grant Access Using Hierarchies: Enabled only where needed.
- Verification: Tested access using various user roles and profiles.

10. Sharing Rules

Sharing rules extend data access beyond OWD, ensuring that stakeholders receive the right insights without compromising security.

- Criteria-Based Sharing: Emergency alerts shared with Emergency Services.
- Owner-Based Sharing: Records shared upward through the role hierarchy.
- Public Groups: Configured for logistics, retail, and agriculture partners.
- Rule Testing: Conducted with multiple test records for accuracy.
- Audit Tracking: Documented all sharing rules for compliance.
- Quarterly Review: Established review cycle to keep rules updated.



11. Login Access Policies

Login access policies provide additional security by defining conditions under which users can log in. This ensures strong security posture for SkyCast.

- IP Restrictions: Enforced trusted ranges for admins and API users.
- Login Hours: Limited non-admin users to business hours.
- MFA: Mandatory for all admins and privileged accounts.
- SSO Integration: Configured for corporate authentication.
- Login-As Policy: Enabled for admin troubleshooting with auditing.
- Monitoring: All login attempts logged and reviewed regularly.

12. Dev Org Setup

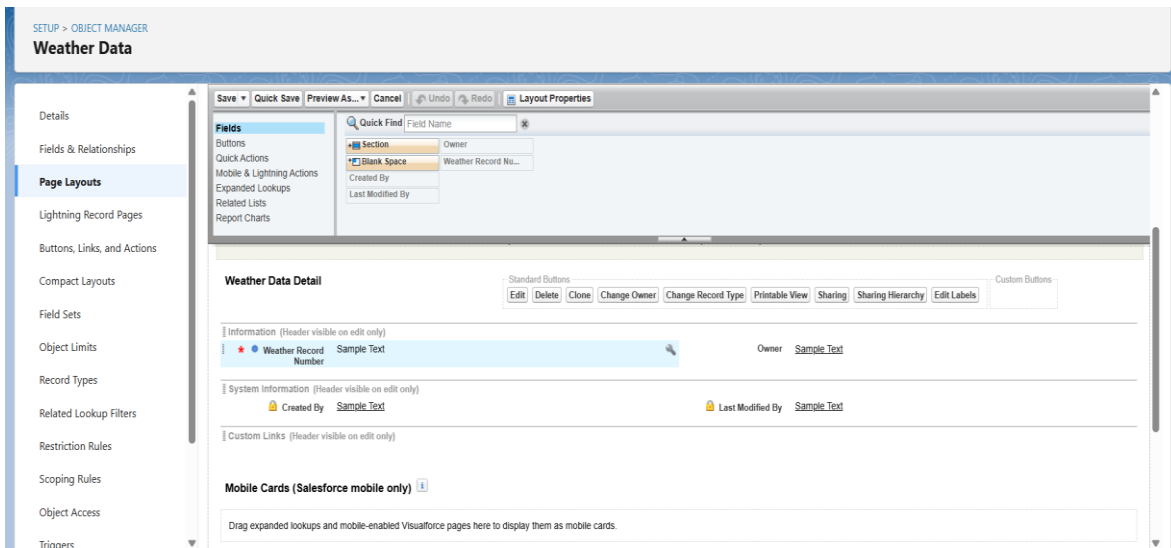
A developer org was established to test and validate configurations before moving to higher environments. This sandboxed approach ensures quality and reduces risks.

- Custom Objects: Created for Weather Data, Alerts, and IoT Readings.
- Page Layouts: Designed separately for different user groups.
- Validation Rules: Implemented to ensure data quality.
- Dashboards: Built sample weather monitoring dashboards.
- Debug Logs: Enabled to monitor integration behavior.
- Training: Used the Dev Org for user training and demonstrations.

The screenshot shows the Salesforce Setup interface for the 'Weather Data' custom object. The breadcrumb trail at the top reads 'SETUP > OBJECT MANAGER'. The page title is 'Weather Data'. On the left, a sidebar lists various configuration options: Details (selected), Fields & Relationships, Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, Field Sets, Object Limits, Record Types, Related Lookup Filters, Restriction Rules, Scoping Rules, Object Access, and Triggers. The main content area is titled 'Details' and contains the following information:

Details	
Description	
API Name	Weather_Data__c
Custom	✓
Singular Label	Weather Data
Plural Label	Weather Data
Enable Reports	✓
Track Activities	✓
Track Field History	✓
Deployment Status	Deployed
Help Settings	Standard salesforce.com Help Window

At the top right of the details section, there are 'Edit' and 'Delete' buttons.



13. Sandbox Usage

Sandboxes were configured to ensure that development, testing, and deployment activities are isolated and safe. This protects the production environment.

- Developer Sandbox: Used for individual unit testing.
- Partial Sandbox: Contained sample data for integration testing.
- Full Sandbox: Created for UAT and performance testing.
- Refresh Strategy: Defined as monthly or after each release cycle.
- Data Masking: Applied to sensitive customer information.
- Deployment Testing: Verified deployment steps in sandbox before production.

14. Deployment Basics

Deployment planning was conducted to standardize movement of configurations across environments. This ensured smooth and error-free releases.

- Release Management: Defined process for Dev → UAT → Prod migration.
- Change Sets: Used for controlled metadata deployment.
- Version Control: GitHub maintained as the repository for metadata.
- Rollback Plans: Prepared for all major deployments.
- Naming Conventions: Implemented to standardize components.
- Documentation: Maintained logs for every deployment cycle.

Phase 3: Data Modeling & Relationships

This phase focuses on designing data structures, relationships, and layouts to manage weather information efficiently in Salesforce.

1. Standard & Custom Objects

Overview:

Salesforce provides Standard Objects (Account, Contact, User, etc.) and allows creation of Custom Objects to handle project-specific data.

For this project, all weather-related objects are custom.

Objects Implemented:

Object Name Type Description

City__c Custom Stores city information including coordinates

Weather_Record__c Custom Stores weather data (Temperature, Humidity, Pressure, linked City)

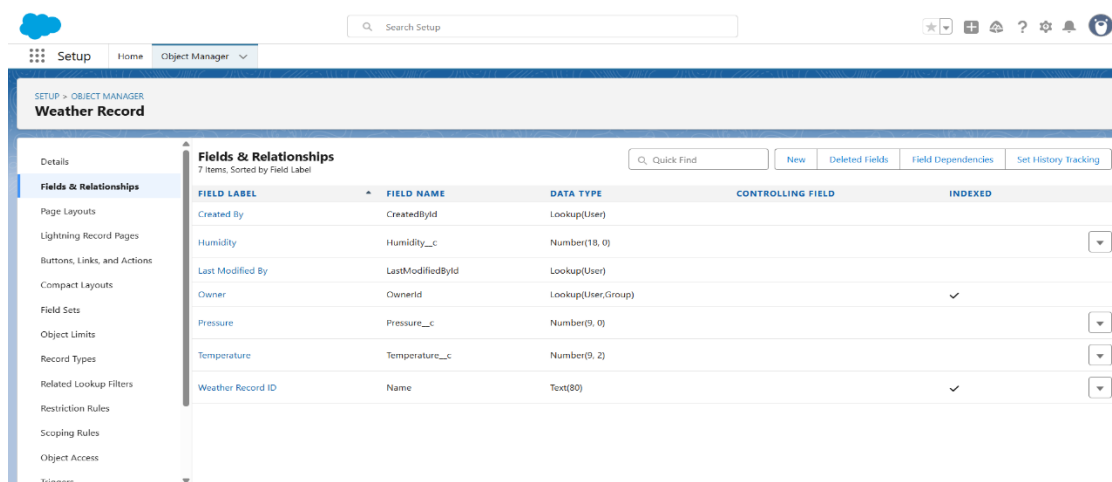
Weather_Alert__c Custom Stores alerts for extreme weather conditions

Weather_Alert_City__c Custom Junction object to enable many-to-many relationship between Alerts and Cities

Optional: External_Weather_Data__x External Connects to OpenWeatherMap API to fetch live weather data

Implementation Notes:

All objects include standard audit fields: Created By, Last Modified By, Owner



The screenshot shows the Salesforce Setup interface for the 'Weather Record' object. The 'Fields & Relationships' tab is selected, displaying a table of fields. The table has columns for Field Label, Field Name, Data Type, Controlling Field, and Indexed. The fields listed are: Created By (Lookup(User)), Humidity (Number(18, 0)), Last Modified By (Lookup(User)), Owner (Lookup(User, Group)), Pressure (Number(9, 0)), Temperature (Number(9, 2)), and Weather Record ID (Text(80)).

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Created By	CreatedById	Lookup(User)		
Humidity	Humidity__c	Number(18, 0)		
Last Modified By	LastModifiedById	Lookup(User)		
Owner	OwnerId	Lookup(User, Group)		✓
Pressure	Pressure__c	Number(9, 0)		
Temperature	Temperature__c	Number(9, 2)		
Weather Record ID	Name	Text(80)		✓

2. Fields

City__c Fields:

Field Name Type Description Real-Time Use Case

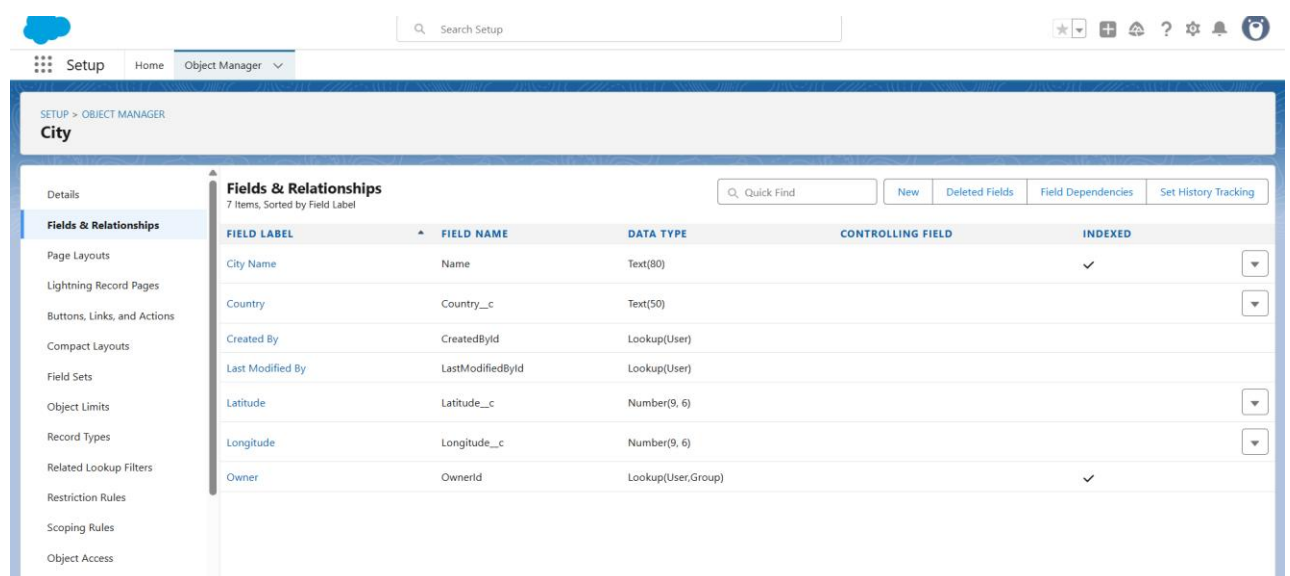
City Name Text Name of the city Identify city in Weather Records

Country__c Text Country of the city For geographical reporting

Latitude__c Number Latitude coordinate For mapping and external API calls

Longitude__c Number Longitude coordinate For mapping and external API calls

Weather_Record__c Fields:



The screenshot shows the Salesforce Setup interface for the 'City' object. The 'Fields & Relationships' section is active, displaying a table of fields. The table has columns for Field Label, Field Name, Data Type, Controlling Field, and Indexed. The fields listed are: City Name (Text(80), indexed), Country (Text(50)), Created By (Lookup(User)), Last Modified By (Lookup(User)), Latitude (Number(9, 6)), Longitude (Number(9, 6)), and Owner (Lookup(User, Group)).

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
City Name	Name	Text(80)		✓
Country	Country__c	Text(50)		
Created By	CreatedById	Lookup(User)		
Last Modified By	LastModifiedById	Lookup(User)		
Latitude	Latitude__c	Number(9, 6)		
Longitude	Longitude__c	Number(9, 6)		
Owner	OwnerId	Lookup(User, Group)		✓

Key Field Considerations

1. **Essential Data Capture** – The fields are carefully designed to store vital weather and city information such as City Name, Temperature, Humidity, Pressure, and Coordinates, ensuring the system can manage both geographical and climate-related data effectively.
2. **Practical Usage** – Every field supports a real-time business use case. For example, Temperature and Humidity fields drive dashboard insights, Precipitation fields help in flood alerts, and Timezone ensures weather records are logged accurately for each city.
3. **Data Accuracy & Integrity** – Relationships like Lookup and Master-Detail guarantee that every Weather Record is tied to the correct City, maintaining consistency and preventing duplicate or orphaned records.
4. **Scalability & Analytics** – Advanced fields such as Wind Speed, Visibility, and Condition are included to allow deeper analysis, support predictive weather models, and ensure the system can easily adapt to future requirements without major redesign.

3.Record Types

Currently using default record type for all objects. Optional future enhancement: Different alert types (e.g., Rain Alert, Heatwave Alert) can use Record Types to control page layouts and picklist values.

- All objects (City__c, Weather_Record__c, Weather_Alert__c) currently use the **default record type**, ensuring consistent layouts and access to fields.
- Record Types allow different **page layouts and picklist values** based on record type, supporting tailored business processes.
- **Future enhancements** may include specialized alert types (e.g., Rain Alert, Heatwave Alert) with custom layouts and picklists to improve usability and reporting.
- Benefits include **better user experience**, reduced data entry errors, and **scalable reporting** for different alert categories.

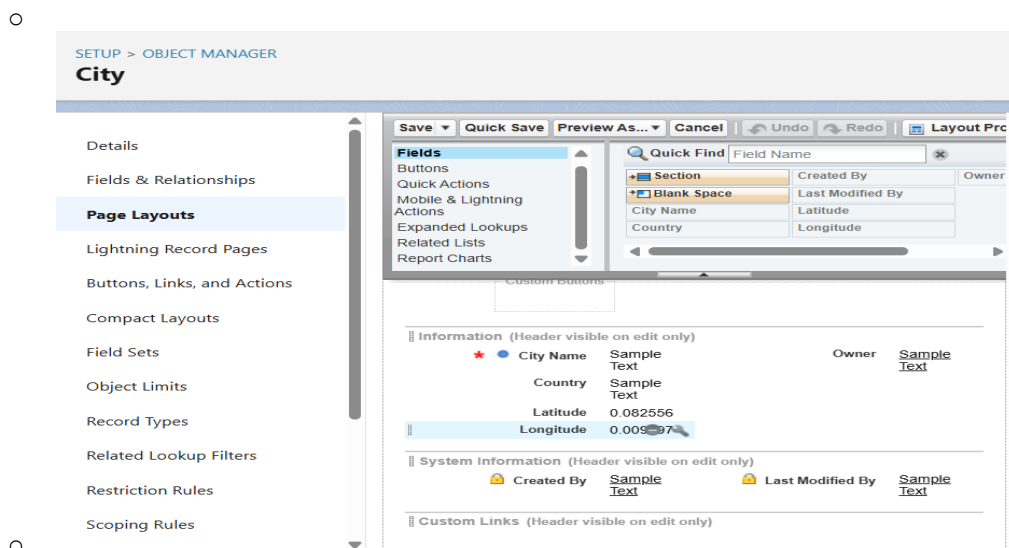
4.Page Layouts

City Page Layout:

- **Fields included:**
 - City Name
 - Country
 - Latitude
 - Longitude

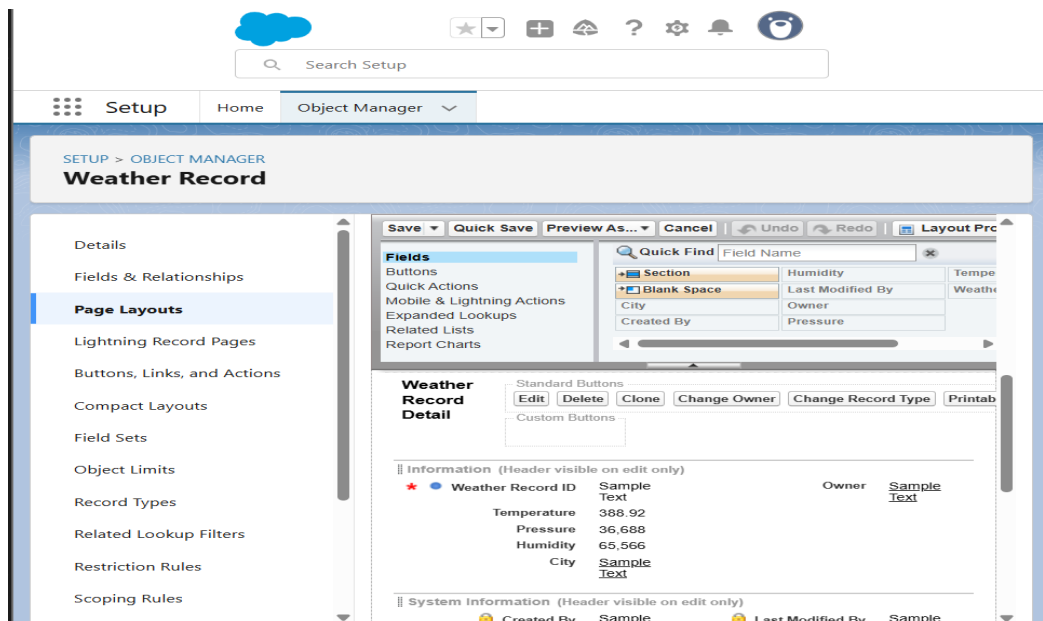
Layout Design:

- Sections organized for **Quick View** and **System Info**.
- Includes **Related Lists** (Weather Alerts) for easy navigation.



Weather Record Page Layout:

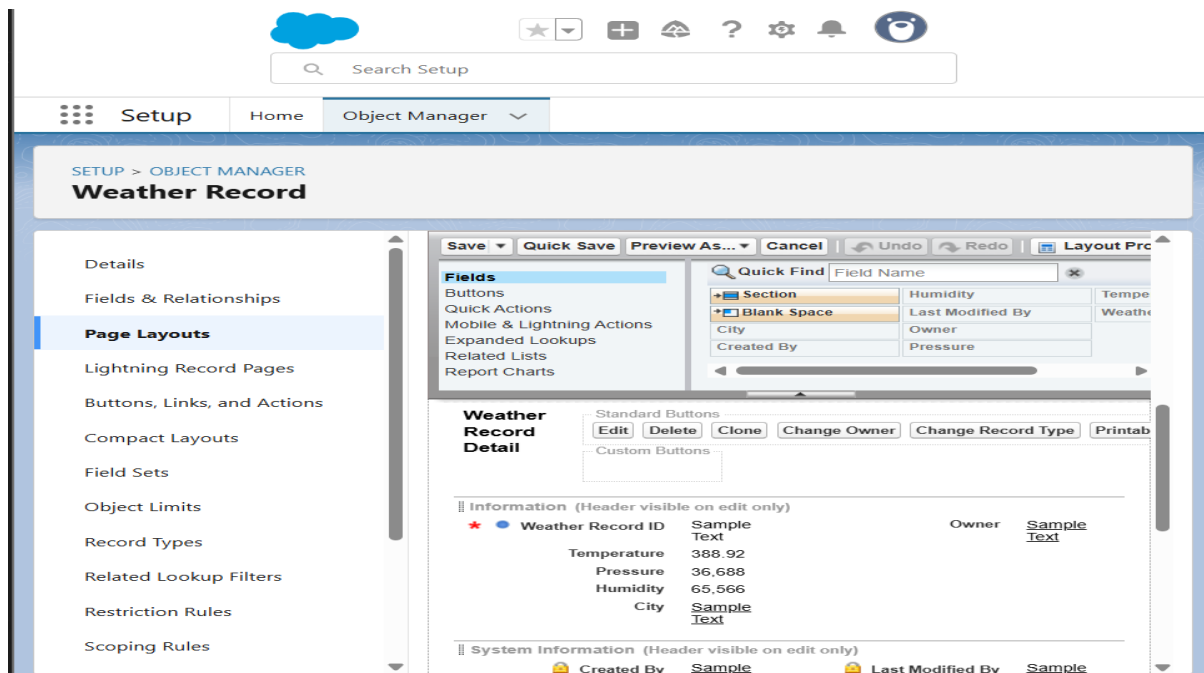
- **Fields included:**
 - City
 - Temperature
 - Humidity
 - Pressure
 - Weather_Record_ID
- **Layout Design:**
 - Sections organized for **Quick View** and **System Info**.
 - Includes **Related Lists** (Weather Alerts) for easy navigation.



5. Compact Layouts

- **Purpose:** Displays essential fields in the Lightning Highlights Panel for quick viewing.
- **City__c Fields Shown:** City Name, Country, Latitude, Longitude.
- **Weather_Record__c Fields Shown:** City, Temperature, Humidity, Weather_Record_ID.
- **Primary Assignment:** Each object has a primary compact layout assigned for consistent record previews.
- **User Benefit:** Enables quick access to critical information without opening the full record page.
- **Lightning Compatible:** Optimized for both desktop and mobile Lightning Experience.

Temperature



6.Schema Builder

Purpose:

- Provides a **visual overview of all objects and their relationships** in the project.
- Helps admins and developers **understand how data flows** between objects.

Relationships Implemented:

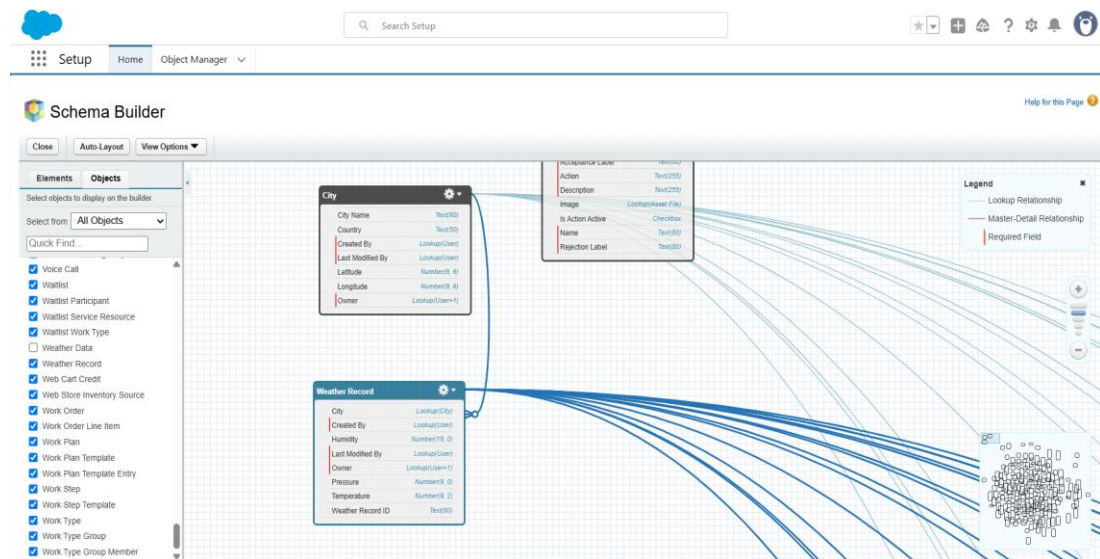
- **Weather_Record__c** → **City__c** (Lookup or Master-Detail) to link weather data with its city.
- **Weather_Alert_City__c** → **Weather_Alert__c** (Master-Detail) and → **City__c** (Master-Detail) to enable many-to-many alerts for multiple cities.

Layout & Organization:

- Objects are arranged for clarity: **City at the top**, Weather Records below, Alerts on the side.
- Drag-and-drop is used to organize the schema for **easy visualization and comprehension**.

Benefits:

- Simplifies understanding of **object relationships** and data dependencies.
- Supports **reporting, dashboards, and future enhancements**.
- Provides a clear reference for **new team members or admins**



7. Lookup vs Master-Detail vs Hierarchical Relationships

• Lookup Relationship:

- Example: Weather_Record__c → City__c
- Weather record exists independently of the city; deleting a city does not delete linked weather records.
- Provides flexibility and optional linking between objects.

• Master-Detail Relationship:

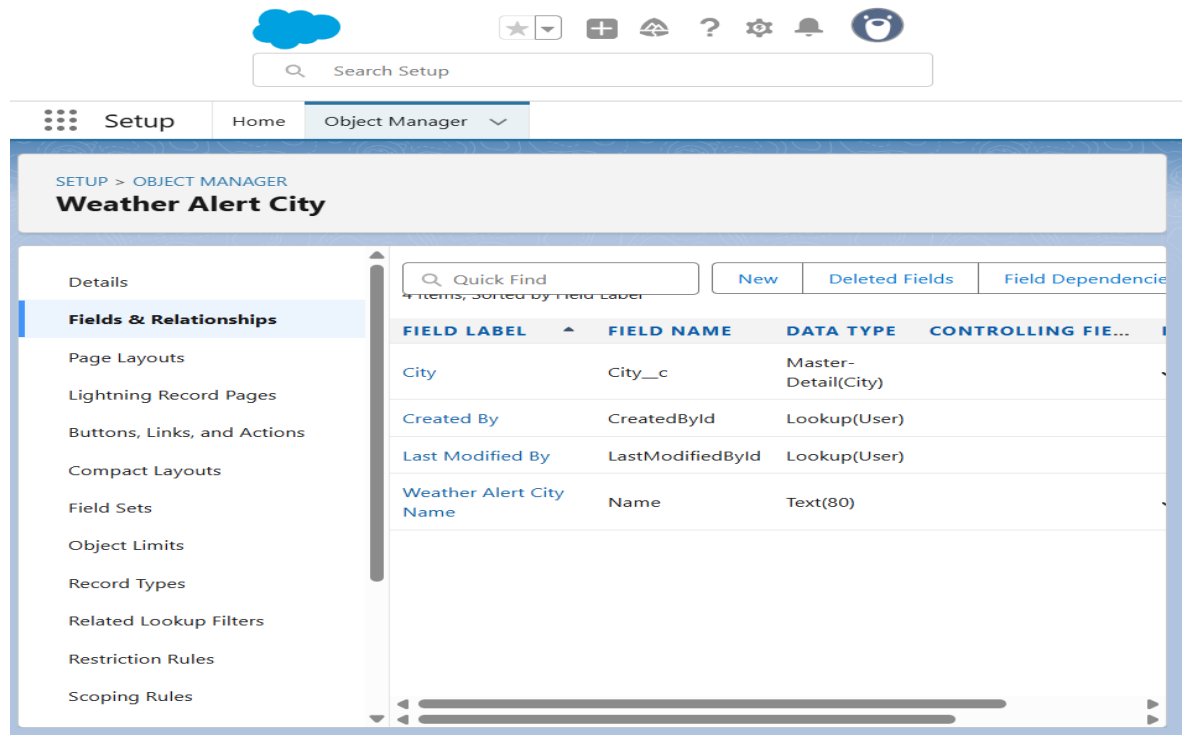
- Example: Weather_Record__c → City__c
- Strong dependency; deleting the parent (City) deletes all child records.
- Supports roll-up summary fields (e.g., average temperature per city).

• Master-Detail (Junction):

- Example: Weather_Alert_City__c → Weather_Alert__c / City__c
- Enables **many-to-many relationships**, allowing multiple alerts for multiple cities.

• Hierarchical Relationship:

- Example: User → Manager
- Not used in the Weather Project but useful for **approval processes and organizational hierarchy**.



8. Junction Objects

- **Purpose:**

- Enables **many-to-many relationships** between Weather Alerts and Cities.
- Allows a single alert to be associated with multiple cities and vice versa.

- **Implementation:**

- Custom object **Weather_Alert_City__c** acts as a junction.
- Connected to **Weather_Alert__c** and **City__c** using Master-Detail relationships.

- **Real-Time Functionality:**

- Multiple cities can be linked to a single alert.
- Multiple alerts can be linked to a single city.
- Ensures accurate tracking and reporting of alert coverage across locations.

- **Benefit:**

- Supports **scalable alert management** across multiple cities.
- Maintains **accurate relationships** between alerts and locations.
- Facilitates **reporting, dashboards, and real-time monitoring** of alert coverage.

9. External Objects

External objects allow Salesforce to **access and display data from external systems** without storing it in Salesforce. In the Weather Project, the external object **External_Weather_Data__x** connects to a service like **OpenWeatherMap API** to fetch live weather data.

- Fields such as **Temperature, Humidity, and Pressure** are mapped from the external system to Salesforce.
- Data is displayed in **Lightning Record Pages or Dashboards**, providing **real-time insights**.
- This approach reduces **Salesforce storage usage** while keeping information **up-to-date**.

Key Benefits:

- Enables **dynamic and real-time monitoring** of weather conditions.
- Reduces internal storage requirements.
- Supports **analytics, dashboards, and decision-making** based on current weather data.

Phase 4 – Process Automation (Admin)

Process Automation in Salesforce streamlines business operations, reduces manual effort, and ensures data consistency. This phase focuses on building robust, real-time solutions using Salesforce’s native automation tools.

1. Validation Rules

Purpose

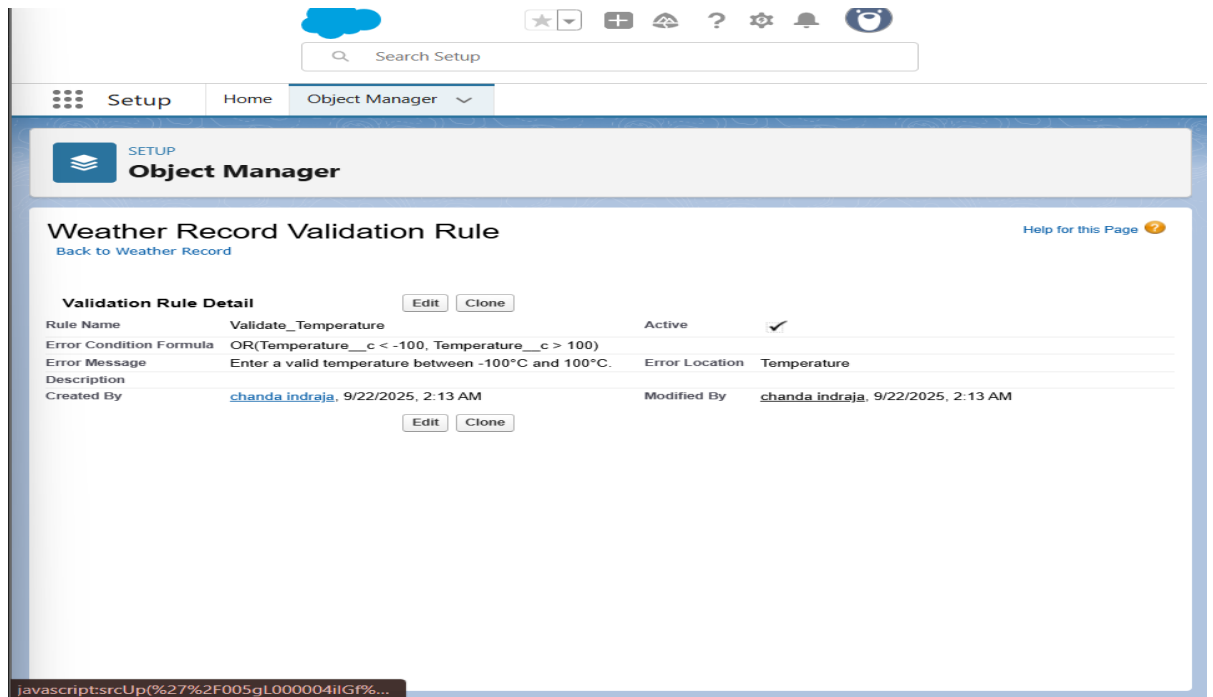
Validation Rules enforce data quality by preventing a record from being saved if it fails specified business criteria. They run whenever a record is created or updated—via the UI, API, Data Loader, or automation—ensuring that only complete and correct data enters Salesforce.

Key Benefits

- **Data Integrity:** Stops inaccurate or incomplete records before they impact reports or dashboards.
- **Compliance:** Enforces regulatory or internal policies (e.g., mandatory fields, date limits).
- **User Guidance:** Provides immediate, actionable error messages to end users.

Real-Time:

Scenario: Sales reps must set the Opportunity **Close Date** to today or a future date.
Validation Formula: `CloseDate < TODAY()`



2. Workflow Rules

Purpose

Workflow Rules automate standard internal processes, allowing Salesforce to perform actions automatically when specific criteria are met.

They help reduce manual effort, improve response time, and ensure consistent business operations.

Key Benefits

- **Efficiency:** Automates routine tasks like sending emails or updating fields, freeing users to focus on high-value work.
- **Consistency:** Ensures that business rules are applied uniformly across all records.
- **Scalability:** Handles large volumes of records with minimal administrative overhead.

The screenshot displays the Salesforce Setup interface for Workflow Rules. The left sidebar shows the navigation menu with 'Workflow Rules' highlighted. The main content area shows the configuration for a specific workflow rule.

Workflow Rule Detail

Field	Value
Rule Name	Severe_Weather_Email_Alert
Object	Weather Record
Active	☒
Evaluation Criteria	Evaluate the rule when a record is created, and every time it's edited
Description	
Rule Criteria	Weather Record: Severe Weather EQUALS TRUE
Created By	chanda.indraja, 9/22/2025, 2:34 AM
Modified By	chanda.indraja, 9/22/2025, 2:38 AM

Workflow Actions

Immediate Workflow Actions

Type	Description
Email Alert	Send Severe Weather Email to Weather Managers

Time-Dependent Workflow Actions [See an example](#)

Message: You cannot add time-dependent workflow actions because your evaluation criteria is "Every time a record is created or edited". [Change Evaluation Criteria](#)

Edit Rule High Value Opportunity Help for this Page ?

Step 3: Specify Workflow Actions Step 3 of 3 Done

Specify the workflow actions that will be triggered when the rule criteria are met. [See an example](#)

Rule Criteria: (Opportunity: Stage NOT EQUAL TO Closed Won) AND (Opportunity: Amount GREATER THAN 1,000,000)
 Evaluation Criteria: Evaluate the rule when a record is created, and any time it's edited to subsequently meet criteria

Immediate Workflow Actions

Action	Type	Description
Edit Remove	Email Alert	Notify Account Team stating that a new high value opportunity has been created
Edit Remove	Email Alert	Notify Executive Committee
Edit Remove	Field Update	Change Probability to 20%
Edit Remove	Outbound Message	Update Marketing Systems

[Add Workflow Action](#)

Time-Dependent Workflow Actions [See an example](#)

14 Days Before Opportunity: Close Date [Edit | Delete](#)

Action	Type	Description
Edit Remove	Task	Opportunity Owner should follow up with customer

[Add Workflow Action](#)

7 Days Before Opportunity: Close Date [Edit | Delete](#)

Action	Type	Description
Edit Remove	Task	Sales Manager should contact the account
Edit Remove	Field Update	Flag for executive involvement

[Add Workflow Action](#)

[Add Time Trigger](#)

3. Process Builder

Purpose

Process Builder allows administrators to build complex, multi-step automation in Salesforce without code.

It supports multiple **if/then** branches and can call **Flows, Apex classes, or other processes**, enabling powerful end-to-end automation.

Real-Time Business Example – Rain Alert Level

Scenario: When a Weather Record indicates a high probability of rain, the system automatically sets the Alert Level to “High.”

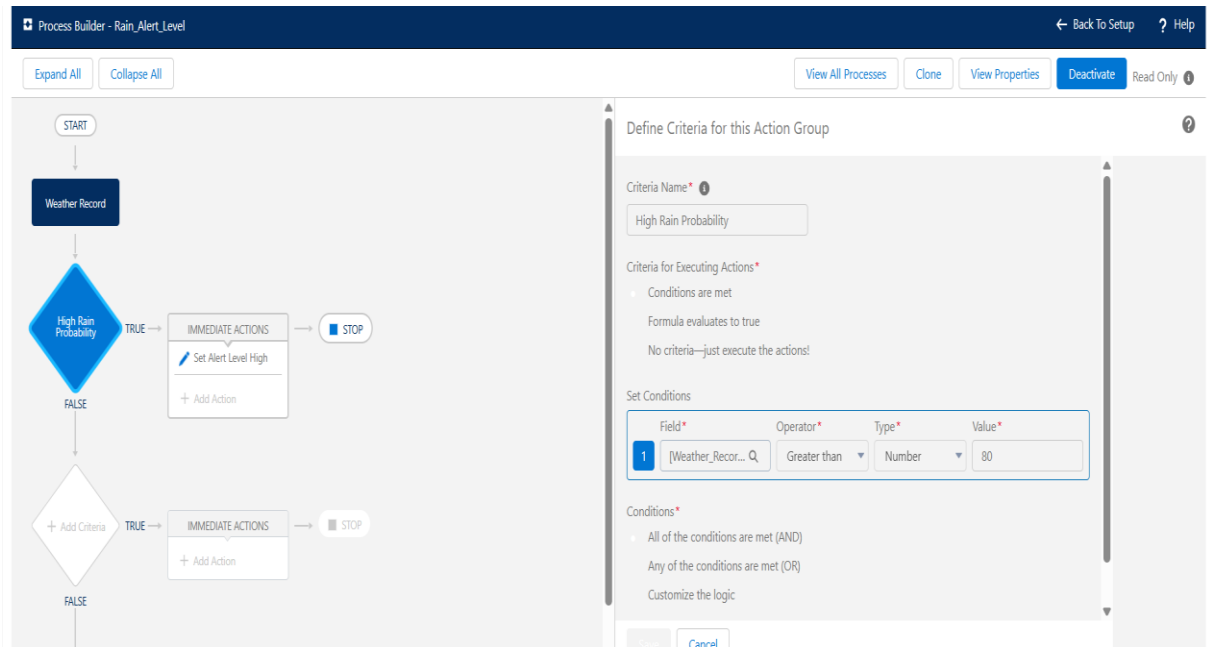
Key Configuration :

- **Object:** Weather Record
- **Criteria Name:** High Rain Probability
- **Condition:** [Weather_Record__c].Rain_Probability__c > 80
- **Immediate Action:** Update the Alert Level field to “High”

Core Features

- **Multiple Criteria Nodes:** Create separate decision branches for different conditions (e.g., Moderate or Low Rain Probability)

- **Immediate & Scheduled Actions:** Trigger field updates, email alerts, tasks, or invoke flows.
- **Integration Options:** Call an Apex class or launch an auto-launched flow for advanced logic.



The screenshot shows the Salesforce Lightning interface for a record named "City Hyderabad". The record is owned by "chanda.indraja". The details section shows the following information:

Related	Details
City Name	Hyderabad
Country	India
Latitude	17.385000
Longitude	78.486700
Created By	chanda.indraja, 9/22/2025, 3:36 AM
Last Modified By	chanda.indraja, 9/22/2025, 3:36 AM

The Activity section shows a list of activities. The filters are set to "All time", "All activities", and "All types". The activity list is currently empty, with the message "No activities to show. Get started by sending an email, scheduling a task, and more." and "No past activity. Past meetings and tasks marked as done show up here."

4. Approval Process

Purpose

An Approval Process in Salesforce automates the routing of records to one or more users for review and sign-off.

It ensures that critical changes—such as pricing overrides, major discounts, or severe weather alerts—are properly authorized before finalization.

Real-Time Example – Severe Weather Approval

Business Requirement: When a Weather Record indicates **Severe Weather = TRUE**, it must be approved by the designated approver before any updates are allowed.

Key Configuration:

- **Process Name:** Severe_Weather_Approval
- **Entry Criteria:** Weather_Record__c.Severe_Weather__c = TRUE
- **Initial Submitter:** City Owner
- **Record Editability:** Administrator ONLY during approval
- **Approval Assignment Email Template:** Severe Weather Alert

Key Features

- **Record Locking:** Prevents edits while a record is under review.
- **Multi-Step Approvals:** Add sequential or parallel approval steps.
- **Automated Field Updates:** Update status fields (Pending, Approved, Rejected) automatically.
- **Notifications:** Email alerts to approvers and submitters.
-

The screenshot shows the Salesforce Setup interface for the 'Severe_Weather_Approval' process. The left sidebar contains navigation links for Setup, Home, and Object Manager. The main content area is titled 'Approval Processes' and shows the details for the 'Weather Record: Severe_Weather_Approval' process. The process definition details include the process name, unique name, description, entry criteria, record editability, approval assignment email template, and initial submitters. The initial submission actions table lists actions like Record Lock, Field Update, and Update Status. The approval steps table shows the process flow with steps, names, descriptions, criteria, assigned approvers, and reject behaviors.

Action	Type	Description
Record Lock	Record Lock	Lock the record from being edited
Update Status Pending	Field Update	Update Status Pending
Update Status NotSubmitted	Field Update	Update Status NotSubmitted
Update Status Approved	Field Update	Update Status Approved
Update Status Rejected	Field Update	Update Status Rejected

Action	Step Number	Name	Description	Criteria	Assigned Approver	Reject Behavior
Step 1	1	Step 1			User: chanda.indraja	Final Rejection

5. Flow Builder (Screen, Record-Triggered, Scheduled, Auto-launched)

Purpose:

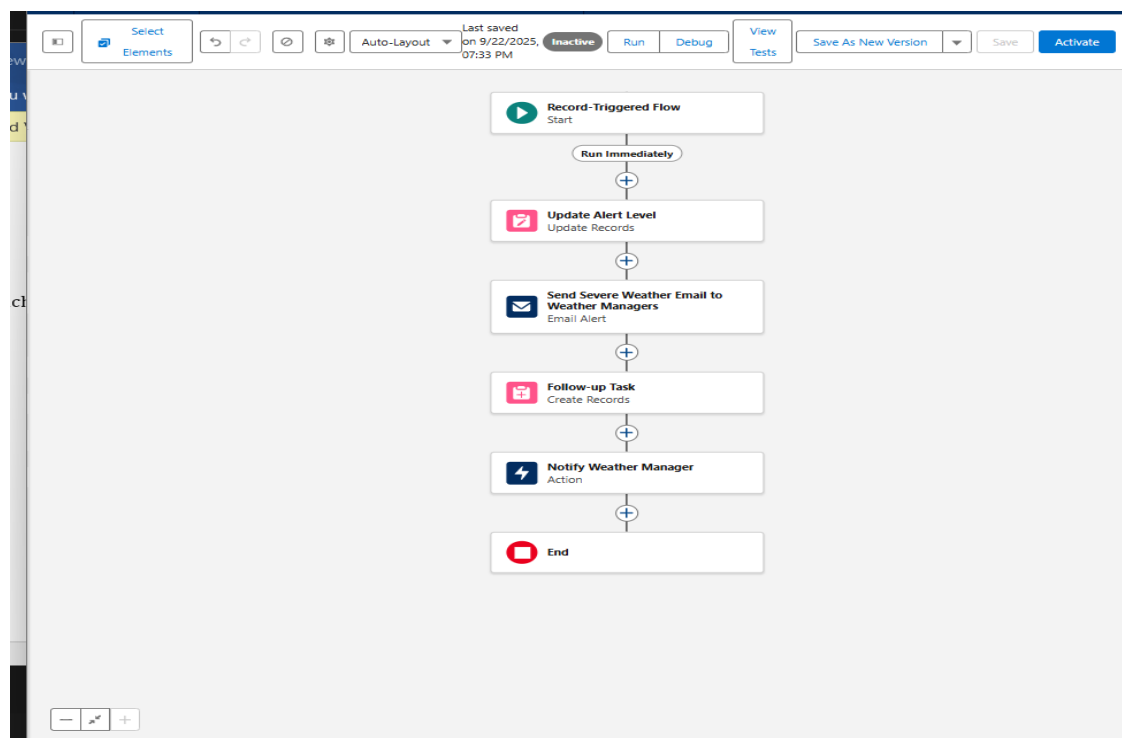
Flow Builder lets you automate business processes without code. It's flexible and can handle complex logic.

Types of Flows & Examples:

1. **Screen Flow** – Guides users step-by-step through a task.
Example: Wizard to create a Case.
2. **Record-Triggered Flow** – Runs automatically when a record is created or updated.
Example: Assign tasks when a new Lead is added.
3. **Scheduled Flow** – Runs at a specified time.
Example: Send monthly renewal reminders.
4. **Auto-Launched Flow** – Runs in the background, often triggered by another process.
Example: Update child records when a parent record changes..

Key Features:

- Automate tasks across Salesforce.
- Send **email notifications** to users.
- Update records, create new records, and perform calculations.
- Conditional logic and loops supported.



6.Email Alerts

Purpose:

Automatically send email notifications based on specific actions or criteria, without manual intervention.

Key Features:

- Uses **Email Templates** (Classic or Lightning) to standardize messages.
- Can be triggered by **Workflow Rules**, **Process Builder**, or **Flow Builder**.
- Can notify multiple recipients: Users, Roles, Contacts, or email addresses.
- Supports **dynamic fields** to personalize emails (e.g., customer name, order number).
- Works with **approvals**, **case updates**, **lead assignment**, and more.

Real-Time Examples:

- Notify a customer when an order is shipped.
- Alert a manager when a high-value opportunity is created.
- Send reminders for overdue tasks or approvals.

The screenshot shows the Salesforce 'Classic Email Templates' setup page. The left sidebar contains a navigation menu with options like 'Email', 'Apex Exception Email', 'Authorized Email Domains', 'Classic Email Templates' (selected), 'Classic Letterheads', 'Compliance BCC Email', 'DKIM Keys', 'Delete Attachments Sent as Links', 'Deliverability', 'Email Address', 'Internationalization', 'Email Attachments', 'Email Delivery Settings', 'Email Domain Filters', 'Email Relays', 'Email Footers', 'Email to Salesforce', and 'Enhanced Email'. The main content area is titled 'Classic Email Templates' and shows details for a 'Text Email Template' named 'Severe Weather Alert'. The template is 'Unfiled Public Classic Email Template' and is 'Available For Use'. The 'Email Template Detail' table lists the template name, unique name, encoding, author, description, and creation/modification details. Below the table, the 'Email Template' section shows the subject line 'Severe Weather Alert for {!Weather_Record__c.Name}' and a 'Plain Text Preview' of the email content, which includes a greeting and weather details for a specific record.

Classic Email Templates

Text Email Template
Severe Weather Alert

Preview your email template below.

Email Template Detail [Edit] [Delete] [Clone]

Email Templates from Salesforce	Unfiled Public Classic Email Templates
Email Template Name	Severe Weather Alert
Template Unique Name	Severe_Weather_Alert
Encoding	Unicode (UTF-8)
Author	chanda indraja [Change]
Description	Email template to notify Weather Managers about severe weather
Created By	chanda indraja, 9/22/2025, 2:20 AM
Modified By	chanda indraja, 9/22/2025, 2:20 AM

[Edit] [Delete] [Clone]

Email Template [Send Test and Verify Merge Fields]

Subject ⚠ Severe Weather Alert for {!Weather_Record__c.Name}

Plain Text Preview

Hello Weather Manager,

Severe weather has been reported for the following weather record: {!Weather_Record__c.Name}.
Details:
- Temperature: {!Weather_Record__c.Temperature__c} °C
- Humidity: {!Weather_Record__c.Humidity__c} %

7.Field Updates

Purpose:

- Maintain data accuracy and consistency.
- Save user time by reducing manual updates.
- Trigger changes that influence other automation (workflows, flows, approval processes).

Use Cases:

- **Opportunity Management:** Automatically update the **Opportunity Stage** to “Closed Lost” when the “Competitor” field is filled in.
- **Case Management:** Update a **Case Status** to “Escalated” when the case priority is set to “High.”
- **Lead Qualification:** Mark a **Lead Source** field to “Web” if the lead was created from a web form.

Benefits:

- Eliminates human errors in repetitive updates.
- Provides real-time record changes.
- Improves reporting accuracy.

8.Tasks

Definition:

Tasks are to-do items that Salesforce can automatically create for users when a specific trigger condition is met. These tasks help guide sales and service teams by ensuring they follow up on critical actions.

Purpose:

- Assign follow-up actions to users at the right time.
- Keep sales cycles moving and avoid missed opportunities.
- Standardize customer engagement processes.

Use Cases:

- **Lead Follow-Up:** When a new high-value lead is created, automatically assign a task for the sales rep to call within 24 hours.
- **Renewals:** When an opportunity is 30 days away from its close date, assign a task for the account manager to reach out to the client.
- **Customer Support:** When a case is reopened, create a task for a support agent to review it immediately.
- Reduces the risk of missed follow-ups.

The screenshot displays the Salesforce Setup interface. On the left, a sidebar contains a search bar with the text 'task' and a list of navigation items: 'Process Automation', 'Workflow Actions', and 'Tasks' (which is highlighted). The main content area is titled 'Tasks' and shows the details for a task named 'Review Weather Record'. This section includes a table for 'Workflow Task Detail' with the following data:

Workflow Task Detail		Edit	Delete	Clone
Object	Weather Record	Status	Not Started	
Assigned To	User : SkyCast Admin	Priority	Normal	
Subject	Review Weather Record			
Unique Name	Review_Weather_Record			
Due Date	Weather Record: Created Date			
Comments				
Created By	chanda.indraja, 9/22/2025, 9:42 AM	Modified By	chanda.indraja, 9/22/2025, 9:42 AM	
	Edit	Delete	Clone	

Below the table, there are two sections: 'Rules Using This Task' and 'Approval Processes Using This Task'. The 'Rules Using This Task' section indicates that this task is currently not used by any rules. The 'Approval Processes Using This Task' section is also visible.

9. Custom Notifications

Purpose:

- Improve response time for urgent matters.
- Keep users informed without relying solely on emails.
- Provide flexibility in how users receive alerts (desktop, mobile).

Use Cases:

- **Sales Alerts:** Notify sales reps instantly when a “Hot Lead” is assigned to them.
- **Case Management:** Alert support managers when a VIP customer’s case is escalated.
- **Approvals:** Send a notification to a manager when a discount approval request is pending.

Phase 5: Apex Programming (Developer)

1.Classes & Objects

Classes in Apex

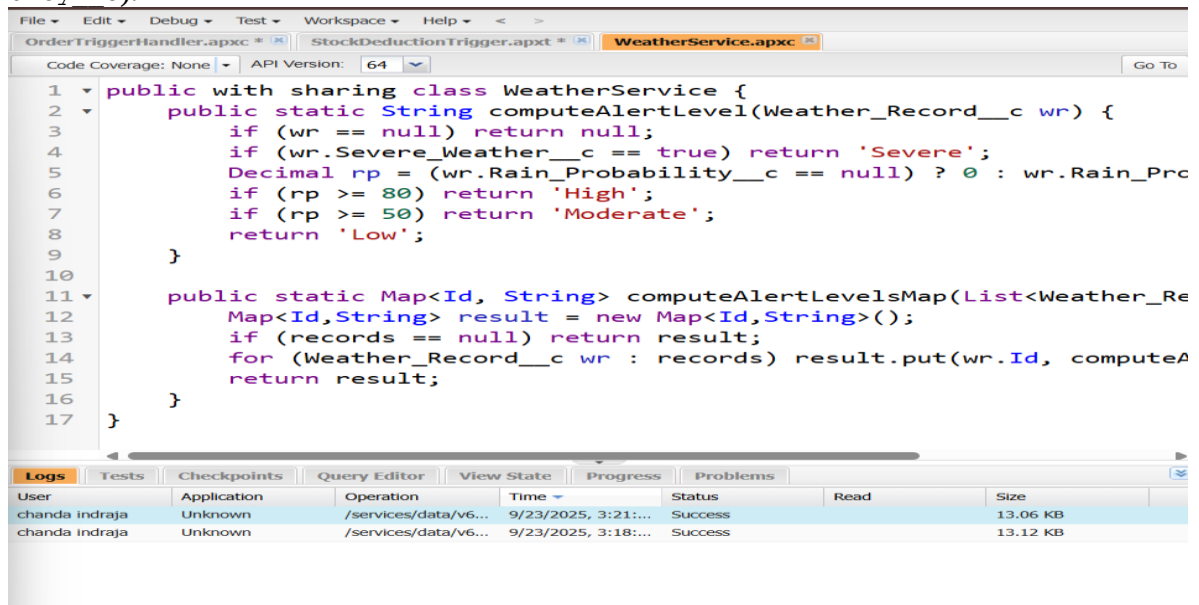
- **Definition:**
Classes are blueprints or templates used to create objects in Salesforce. They encapsulate **business logic**, **data structures**, and **methods** that define the behavior of an object.
- **Purpose:**
 - Centralize code for easier maintenance and reuse.
 - Separate complex logic from triggers to follow **best practices**.
 - Facilitate **unit testing** of business logic independently of UI or triggers.

Key Components of a Class:

- **Properties (Variables):** Hold data (e.g., temperature, humidity).
- **Methods (Functions):** Contain logic to manipulate or fetch data.
- **Constructors:** Special methods used to initialize objects.
- **Access Modifiers:** public, private, global – control visibility.
- **Example – Weather Project Objects:**

Types of Objects:

- **Standard Objects:** Predefined by Salesforce (e.g., Account, Contact).
- **Custom Objects:** Created specifically for your project (e.g., Weather_Record__c, City__c).



```
1 public with sharing class WeatherService {
2     public static String computeAlertLevel(Weather_Record__c wr) {
3         if (wr == null) return null;
4         if (wr.Severe_Weather__c == true) return 'Severe';
5         Decimal rp = (wr.Rain_Probability__c == null) ? 0 : wr.Rain_Probability__c;
6         if (rp >= 80) return 'High';
7         if (rp >= 50) return 'Moderate';
8         return 'Low';
9     }
10
11     public static Map<Id, String> computeAlertLevelsMap(List<Weather_Record__c> records) {
12         Map<Id, String> result = new Map<Id, String>();
13         if (records == null) return result;
14         for (Weather_Record__c wr : records) result.put(wr.Id, computeAlertLevel(wr));
15         return result;
16     }
17 }
```

User	Application	Operation	Time	Status	Read	Size
chanda indraja	Unknown	/services/data/v6...	9/23/2025, 3:21:...	Success		13.06 KB
chanda indraja	Unknown	/services/data/v6...	9/23/2025, 3:18:...	Success		13.12 KB

2.Apex Triggers (before/after insert/update/delete)

1. What is an Apex Trigger

- **Definition:**

An Apex Trigger is a piece of Apex code that executes **before or after a record is inserted, updated, deleted, or undeleted** in Salesforce.

- **Purpose:**

- Automate business processes.
- Perform operations that cannot be done with declarative tools like Workflow or Process Builder.
- Ensure data integrity and enforce complex logic.

2. Trigger Events

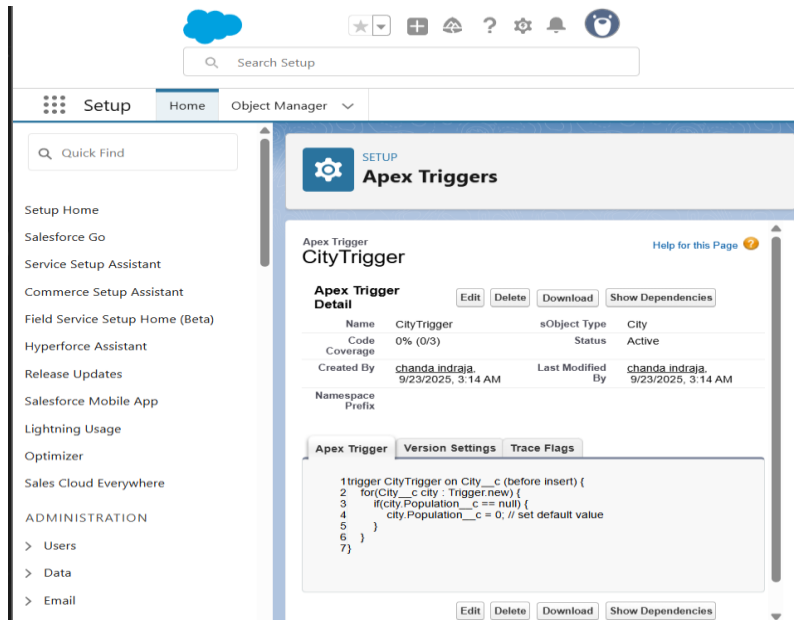
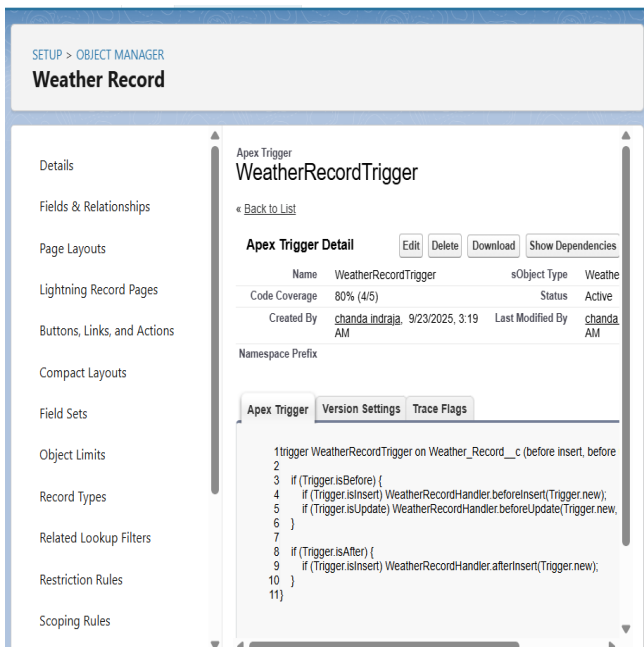
Triggers can be executed **before** or **after** DML (Data Manipulation Language) operations:

1. **Before Triggers:**

- Executed **before** a record is saved to the database.
- Used to **validate or modify field values** before insertion or update.
- Example: Automatically setting default temperature for new weather records.

2. **After Triggers:**

- Executed **after** a record is saved to the database.
- Used for actions that require **record IDs** or interactions with other objects.
- Example: Creating related Weather_Record__c after a City__c is added.



```

File Edit Debug Test Workspace Help < >
WeatherRecord.apxt * WeatherRecordHandler.apxc * Log executeAnonymous @9/23/2025, 3:55:01 PM Log ex
Code Coverage: None API Version: 64 Go To
1 public with sharing class WeatherRecordHandler {
2
3 // BEFORE INSERT
4 public static void beforeInsert(List<Weather_Record__c> newList) {
5     for (Weather_Record__c wr : newList) {
6         if (wr.Temperature__c < -50 || wr.Temperature__c > 70) {
7             wr.addError('Temperature seems invalid. Please check');
8         }
9     }
10 }
11
12 // BEFORE UPDATE
13 public static void beforeUpdate(List<Weather_Record__c> newList) {
14     for (Weather_Record__c wr : newList) {
15         Weather_Record__c oldRec = oldMap.get(wr.Id);
16
17         if (wr.Temperature__c != oldRec.Temperature__c &&
18             (wr.Temperature__c < -50 || wr.Temperature__c > 70)) {
19
20

```

Search...

Sales Home Opportunities Leads Tasks Weather Records More

Weather Record **WR005**

City	Temperature	Pressure
Mumbai	25.00	1,013
Humidity	70	

Weather Forecast

Enter City

mumbai

Get Weather

City: Mumbai
Description: Temperature: 28.99 °C
Humidity: 79 %

3.Trigger Design Pattern

1. What is a Trigger Design Pattern

- **Definition:**
A Trigger Design Pattern is a **structured way to write Apex triggers** that ensures clean, maintainable, and scalable code.
- **Purpose:**
 - Avoid messy triggers with duplicated logic.
 - Handle **bulk operations** efficiently.
 - Separate trigger logic from business logic (delegated to handler classes).

2. Key Components

1. **Trigger:**
 - The entry point that listens to events (insert, update, delete).
 - Should be **slim**; only calls the handler class.
2. **Handler Class:**
 - Contains the main business logic (create, update, delete related records).
 - Bulkified to handle multiple records at once.
3. **Utility or Service Classes (optional):**
 - For API calls, complex calculations, or reusable methods.

3. Benefits

- Easy to maintain and update.

- Single trigger per object.
- Supports bulk operations for multiple records.
- Reduces errors and improves readability.

The screenshot shows an IDE with two tabs: 'WeatherRecord.apxt' and 'WeatherRecordHandler.apxc'. The main editor displays the following Apex code:

```

1 trigger WeatherRecordTrigger on Weather_Record__c (before insert, be
2     if(Trigger.isBefore){
3         if(Trigger.isInsert) WeatherRecordHandler.beforeInsert(Trigg
4         if(Trigger.isUpdate) WeatherRecordHandler.beforeUpdate(Trigg
5
6
7
8     };
9     insert city;
10    System.debug('City created: ' + city.Id);
11
12    // Step 2: Insert a valid Weather_Record
13    Weather_Record__c wrValid = new Weather_Record__c(
14        Temperature__c = 25,
15        Pressure__c = 1013,
16        Humidity__c = 70,
17        City_Master_Detail__c = city.Id // | Corrected Master-Deta
18    );
19    insert wrValid;
20

```

An 'Enter Apex Code' modal window is open, showing the same code as the main editor. The bottom of the IDE shows a 'Logs' tab and a table with columns 'Name', 'Line', and 'Problem'.

4.SOQL & SOSL

1. SOQL (Salesforce Object Query Language)

- **Definition:**
SOQL is used to **query records from Salesforce objects**.
- **Purpose:**
 - Retrieve specific fields and records.
 - Filter and order data using WHERE, ORDER BY, LIMIT.
- **Example – Fetch Weather Records by City:**

```

List<Weather_Record__c> records = [

    SELECT Id, Temperature__c, Humidity__c

    FROM Weather_Record__c

    WHERE City__c = :cityId

];

```

2. SOSL (Salesforce Object Search Language)

- **Definition:**
SOSL is used to **search text, email, and phone fields** across multiple objects at once.
- **Purpose:**
 - Find records when you **don't know the exact field** where the data exists.
 - Useful for **search functionality** in Lightning Web Components (LWC) or Apex.
- **Example – Search Weather Records by Keyword:**

List<List<SObject>> searchResults = [FIND 'rain*' IN ALL FIELDS RETURNING Weather_Record__c(Name, Description__c)];

The screenshot shows the Salesforce IDE interface with the Execution Log open. The log displays several events, including LIMIT_USAGE and SOSL_EXECUTE. A modal window titled "Log line #542 details" is open, showing the following text:

```
16:43:39:024 USER_DEBUG [65]||DEBUG|SOSL Result: ((City__c:{Id=a08gL00000Ab2mgQAB, Name=Mumbai}, City__c:{Id=a08gL00000Ab1tpQAB, Name=Mumbai}, City__c:{Id=a08gL00000Aav24QAB, Name=Mumbai}), ())
```

The modal window has an "OK" button at the bottom.

The screenshot shows the Salesforce IDE interface with the Execution Log open. The log displays several events, including LIMIT_USAGE and SOQL_EXECUTE. A modal window titled "Log line #499 details" is open, showing the following text:

```
16:43:38:654 SOQL_EXECUTE_BEGIN [57]||Aggregations:0|SELECT Id, Last_Updated_Weather__c FROM City__c WHERE Id = :tmpVar1
```

The modal window has an "OK" button at the bottom.

5.Collections: List, Set, Map

Overview

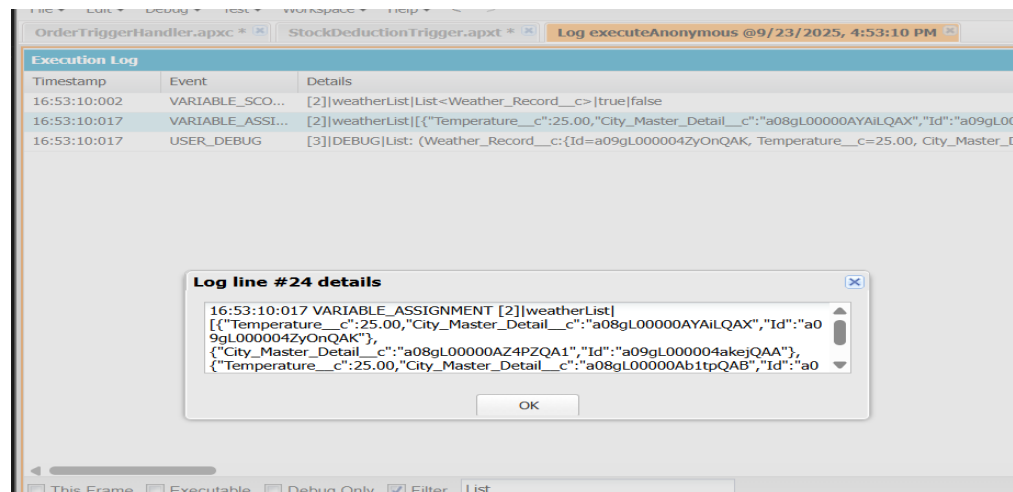
Collections in Apex are variables that can store multiple records or values together. They are widely used for **bulk data handling** in Salesforce.

Types of Collections

- **List**
 - Ordered collection.
 - Can contain duplicate values.
 - Index starts from 0.
 - Example use: Store multiple city names.

```
List<String> cities = new List<String>{'Delhi', 'Mumbai', 'Chennai'};
```

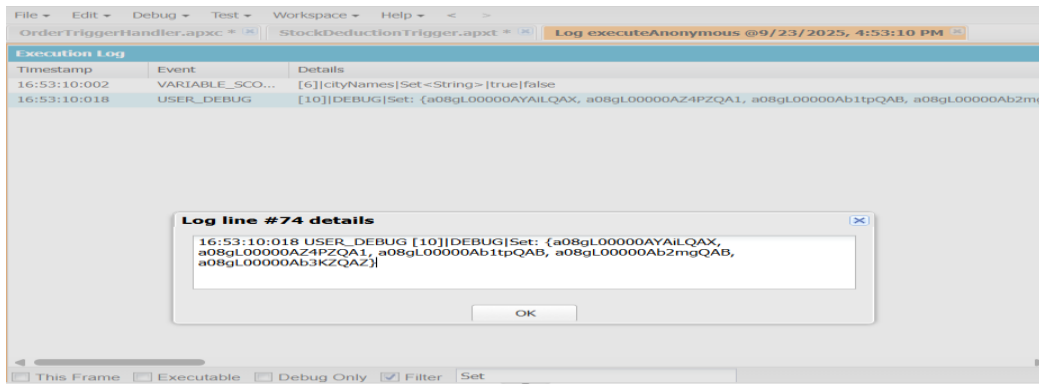
```
System.debug(cities[0]); // Delhi
```



- **Set**
 - Unordered collection.
 - Does not allow duplicate values.
 - Example use: Store unique city names to avoid duplicates.

```
Set<String> cities = new Set<String>{'Delhi', 'Mumbai', 'Delhi'};
```

```
System.debug(cities.size());
```



- **Map**

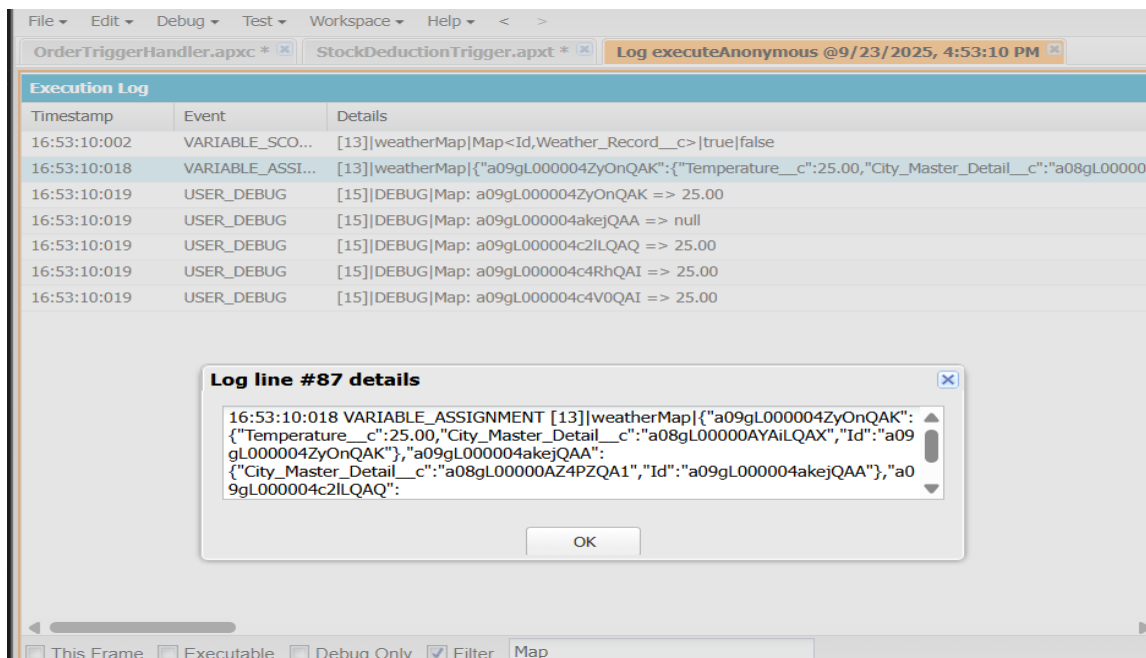
- Collection of key–value pairs.
- Each key is unique, but values can repeat.
- Example use: Map city names to their temperature values.

```
Map<String, Integer> cityTemp = new Map<String, Integer>();
```

```
cityTemp.put('Delhi', 32);
```

```
cityTemp.put('Mumbai', 29);
```

```
System.debug(cityTemp.get('Delhi')); // 32
```



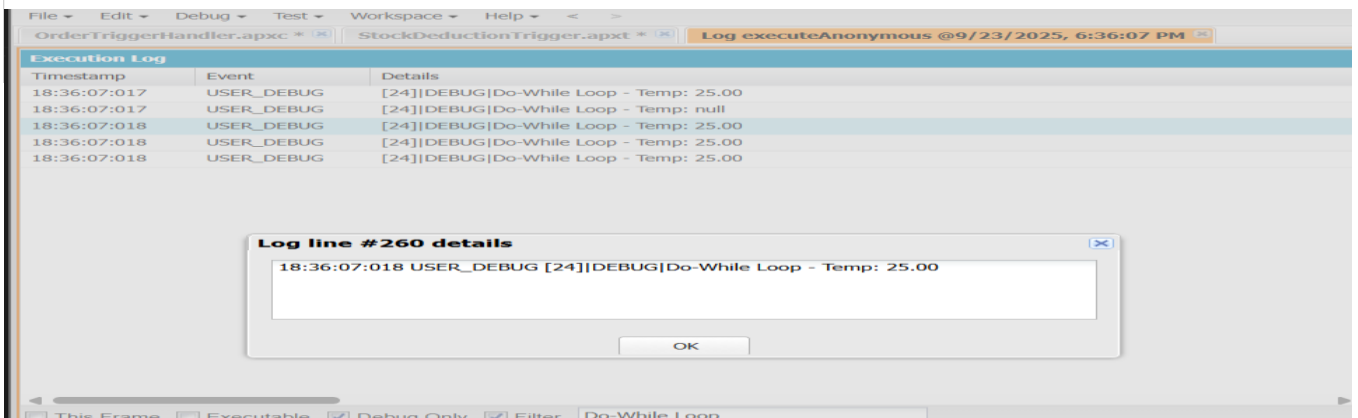
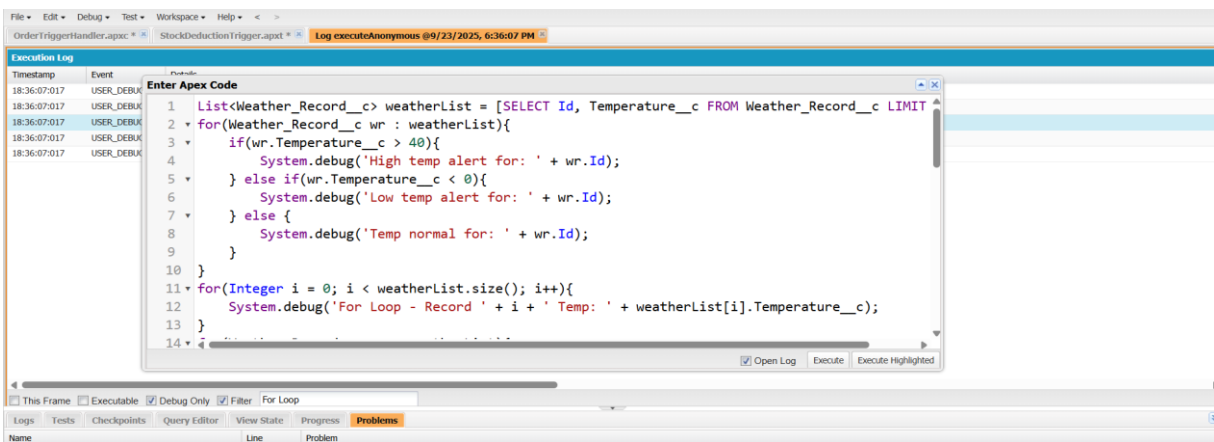
6. Control Statements

Overview

Control statements define the **flow of logic** in Apex programs.

Types

- **Conditional Statements**
 - if-else → Executes code based on conditions.
 - switch → Cleaner alternative to multiple if-else statements.
- **Looping Statements**
 - for → Iterates through records for a fixed number of times.
 - for-each → Simplified loop for collections.
 - while → Repeats as long as condition is true.
- **Break & Continue**
 - break → Immediately exits a loop.
 - continue → Skips current iteration and moves to next



7. Batch Apex

Overview

Batch Apex is used to process **large volumes of data** in manageable chunks, without hitting governor limits.

Why It's Needed

- Salesforce has strict **limits** on processing records.
- Batch Apex splits large jobs into smaller sets and processes them asynchronously.

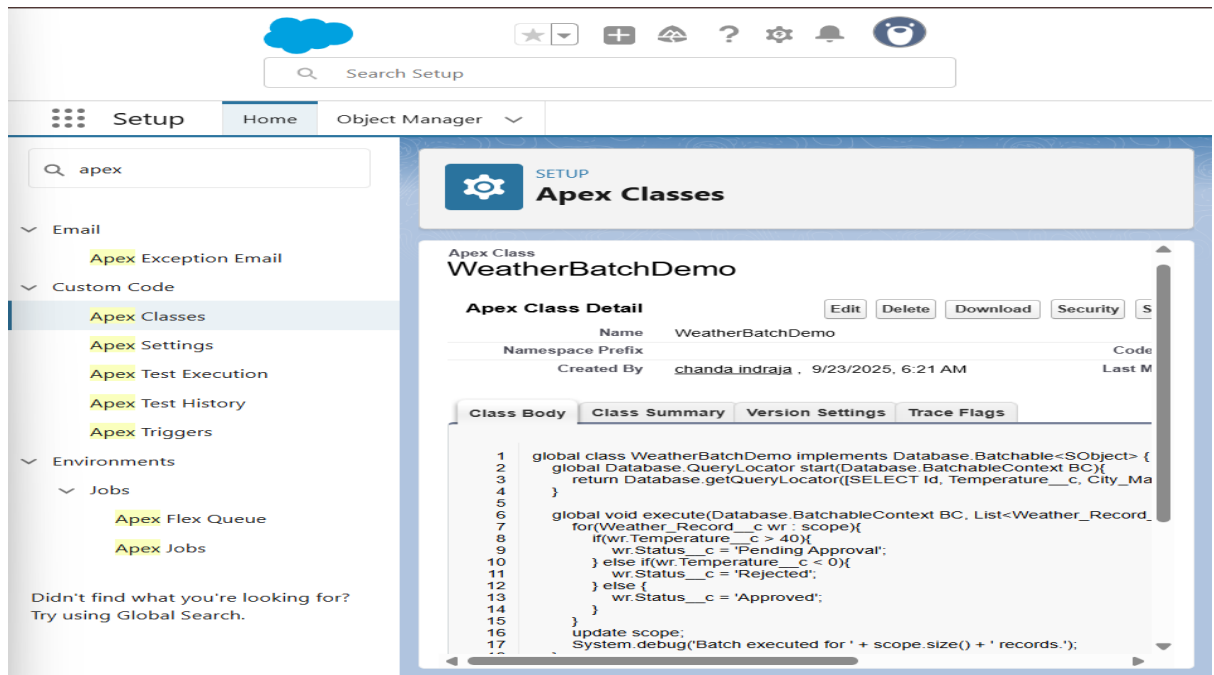
Structure

A Batch Apex class has three main methods:

1. **Start** → Collects the records to be processed.
2. **Execute** → Runs on each batch of records.
3. **Finish** → Performs final tasks once all batches are complete.

Benefits

- Handles millions of records safely.
- Runs in the background (asynchronous).
- Useful for scheduled jobs, data cleanup, and API-based data processing.



The screenshot shows the Salesforce Setup interface. The left sidebar contains a search bar with 'apex' and a list of categories: Email, Custom Code, and Environments. Under Custom Code, 'Apex Classes' is selected. The main content area is titled 'Apex Classes' and shows the details for the 'WeatherBatchDemo' class. The class details include the name, namespace prefix, and the class body code. The class body code is as follows:

```
1 global class WeatherBatchDemo implements Database.Batchable<SObject> {
2     global Database.QueryLocator start(Database.BatchableContext BC){
3         return Database.getQueryLocator([SELECT Id, Temperature__c, City_Ma
4     }
5
6     global void execute(Database.BatchableContext BC, List<Weather_Record_
7         for(Weather_Record__c wr : scope){
8             if(wr.Temperature__c > 40){
9                 wr.Status__c = 'Pending Approval';
10            } else if(wr.Temperature__c < 0){
11                wr.Status__c = 'Rejected';
12            } else {
13                wr.Status__c = 'Approved';
14            }
15        }
16        update scope;
17        System.debug('Batch executed for ' + scope.size() + ' records.');
```


8.Queueable Apex

Overview

Queueable Apex is used to run **asynchronous jobs** in Salesforce. It is similar to `Future` methods but more powerful.

Key Features

- Allows **chaining** of jobs (one queueable can enqueue another).
- Supports **complex data types** like `sObjects` and custom classes as parameters.
- More flexible than `Future` methods.

Benefits

- Handles long-running processes without blocking users.
- Can process large datasets in the background.
- Easy to monitor in the **Apex Jobs** page.
- **Interface Used:** `System.Queueable`.
- Can process **more records than Future methods**, but still within governor limits.
- Can be **monitored, paused, and retried** from the **Apex Jobs** page.
- Supports **complex data types** as parameters (Lists, `sObjects`, Maps).
- Jobs can be **chained** → one job automatically starts another.
- Easier to test compared to `Future` methods.

The screenshot shows the Salesforce Setup interface. On the left, the 'Setup' menu is open, and 'Apex Classes' is selected under 'Custom Code'. The main content area displays the 'Apex Classes' page for the class 'WeatherQueueable'. The page includes a search bar at the top, a navigation bar with 'Setup', 'Home', and 'Object Manager', and a sidebar with a search bar and a list of setup items. The 'Apex Class Detail' section shows the class name 'WeatherQueueable', its namespace prefix, and the creator 'chanda.indraja' on '9/23/2025, 6:40 AM'. Below this, the 'Class Body' tab is active, displaying the following Apex code:

```
1 public class WeatherQueueable implements Queueable {
2     public void execute(QueueableContext context) {
3         List<Weather_Record__c> weatherList = [SELECT Id, Temperature__c, S
4         for (Weather_Record__c wr : weatherList){
5             if (wr.Temperature__c > 40) wr.Status__c = 'Pending Approval';
6             else if (wr.Temperature__c < 0) wr.Status__c = 'Rejected';
7             else wr.Status__c = 'Approved';
8         }
9         update weatherList;
10        System.debug('Queueable executed for ' + weatherList.size() + ' Weather
11    }
12 }
```

9.Scheduled Apex

1. Purpose

- Automates tasks at **specific times or intervals**.
- Removes the need for manual execution of repetitive jobs.
- Useful for **maintenance, integration, and data refreshes**.

2. Features

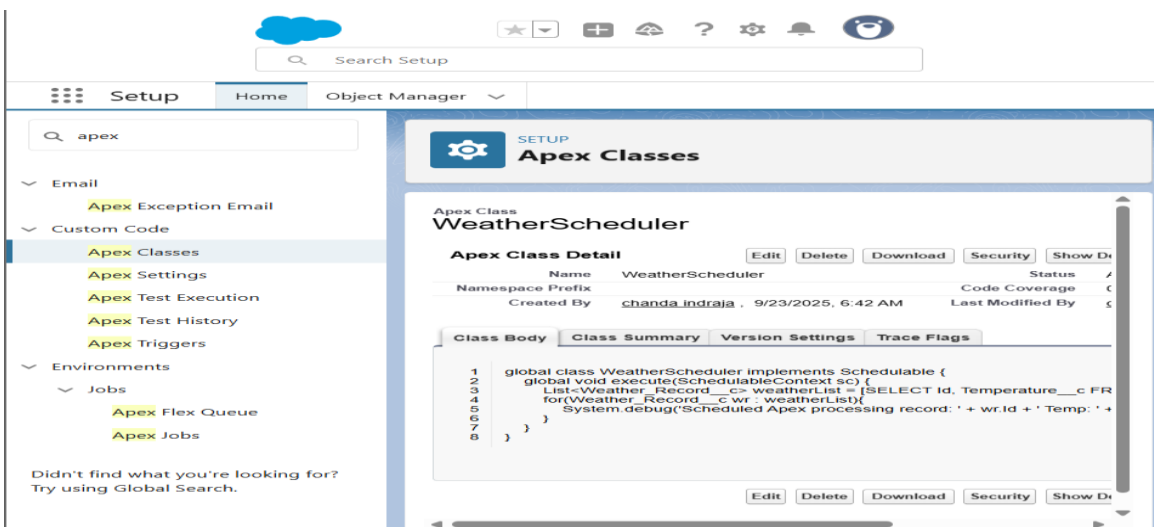
- Runs in the background (asynchronous execution).
- Can be scheduled via **Setup UI** or **System.schedule() method**.
- Uses **cron expressions** to define schedule (similar to UNIX cron jobs).
- Can call **Batch Apex** or **Queueable Apex** inside scheduled jobs.

3. Use Cases

- Daily or nightly updates (e.g., refresh weather data for all cities).
- Weekly or monthly reports (e.g., send summary of weather trends).
- Data cleanup jobs (e.g., remove old or duplicate weather records).
- Scheduled API callouts (e.g., fetch data from external weather API).

4. Cron Expression Format

- Structure: **Seconds Minutes Hours Day_of_Month Month Day_of_Week Year (optional)**
- Example:
 - "0 0 0 * * ?" → Runs daily at midnight.
 - "0 0 12 ? * MON" → Runs every Monday at 12 PM.



The screenshot displays the Salesforce Setup interface. On the left, the 'Setup' menu is visible with 'Apex Classes' selected under 'Custom Code'. The main content area shows the 'Apex Classes' page for a class named 'WeatherScheduler'. The page includes a search bar at the top, a 'Search Setup' field, and a 'Setup' button. The class details section shows the class name 'WeatherScheduler', its namespace prefix, and creation/modification dates. Below this, the 'Class Body' tab is active, displaying the following code:

```
1 global class WeatherScheduler implements Schedulable {
2     global void execute(SchedulableContext sc) {
3         List<Weather_Record__c> weatherList = [SELECT Id, Temperature__c FR
4         for(Weather_Record__c wr : weatherList){
5             System.debug("Scheduled Apex processing record: " + wr.Id + " Temp: " +
6         }
7     }
8 }
```

10.Future Methods

Overview

Future methods are used to run processes **asynchronously** in the background, especially for **callouts to external systems**.

Purpose

- Run operations **asynchronously** (in the background).
- Ideal for long-running tasks or external callouts.

Features

- Annotated with `@future`.
- Executes in a **separate thread** from the main transaction.
- Can be used for **callouts, email sending, and data updates**.

Use Cases

- Making **HTTP callouts** to external systems.
- Performing **resource-intensive calculations**.
- Allows integration with external APIs without blocking user actions.
- Helps avoid governor limits during synchronous execution.

The screenshot displays the Salesforce Setup interface. On the left, a navigation menu shows 'Setup' with a search bar containing 'apex'. Under 'Custom Code', 'Apex Classes' is selected. The main content area is titled 'Apex Classes' and shows details for the 'WeatherFuture' class. The 'Apex Class Detail' section includes fields for Name (WeatherFuture), Namespace Prefix, Created By (chanda.indraja), and Created Date (9/23/2025, 6:43 AM). Below this, the 'Class Body' tab is active, displaying the following Apex code:

```
1 public class WeatherFuture {
2     @future
3     public static void updateWeatherStatus() {
4         List<Weather_Record__c> weatherList = [SELECT Id, Temperature__c, S
5         for(Weather_Record__c wr : weatherList){
6             if(wr.Temperature__c > 40) wr.Status__c = 'Pending Approval';
7             else if(wr.Temperature__c < 0) wr.Status__c = 'Rejected';
8             else wr.Status__c = 'Approved';
9         }
10        update weatherList;
11        System.debug('Future method executed for ' + weatherList.size() + ' Wea
12    }
13 }
```

11. Exception Handling

Overview

Exception handling in Apex is used to **catch errors gracefully** and prevent program crashes.

Types of Exceptions

- **System exceptions:** Thrown by Salesforce platform (e.g., NullPointerException, DmlException).
- **Custom exceptions:** Defined by developers for specific business scenarios.

Mechanism

- Uses `try`, `catch`, and `finally` blocks.
 - **try:** Code that may cause exception.
 - **catch:** Handles the exception.
 - **finally:** Always executes (cleanup).

Benefits

- Prevents application failures.
- Provides user-friendly error messages.
- Ensures smooth transaction rollbacks.

12. Test Classes

1. Purpose

- Validate Apex code functionality.
- Ensure code is **deployable to production** (minimum 75% coverage required).
- Prevent bugs and unintended behavior in real-time usage.

2. Features

- Written using `@isTest` annotation.
- Run in a **test context** (no impact on org data).
- Support **positive, negative, and bulk testing**.

3. Use Cases

- Verifying **trigger logic** when records are inserted or updated.
- Testing **Batch, Queueable, and Future methods**.
- Ensuring **custom validations and exceptions** work as expected.

Apex Test Execution

Click Select Tests to choose one or more Apex unit tests and run them. To see the current code coverage for an individual class or your organization, go to the [Apex Classes](#) page.

Select Tests... Developer Console Options... View Test History

Status	Class	Result				
Test Run: 2025-09-23 09:32:16, chandaindraja119@agentforce.com, (1 test class run)						
✓	(View) WeatherRecordHandlerTest	(1/1) Test Methods Passed				
Test Run: 2025-09-23 09:30:02, chandaindraja119@agentforce.com, (1 test class run)						
Detail	Duration	Class	Method	Pass/Fail	Error Message	Stack Trace
(View)	0:00	WeatherRecordHandlerTest	testWeatherTrigger	Fail	System.DmlException: Insert failed. First exception on row 0; first Class: WeatherRecordHandlerTest.testWeatherTrigger: line 14,	

13. Asynchronous Processing

1. Purpose

- Run long-running or resource-intensive tasks **outside the main transaction**.
- Improves system performance and **avoids hitting governor limits**.

2. Types

- **Future Methods** → Lightweight background tasks.
- **Queueable Apex** → Supports job chaining and monitoring.
- **Batch Apex** → Processes millions of records in chunks.
- **Scheduled Apex** → Runs jobs at defined times.

3. Use Cases

- Making **API callouts** to weather services.
- Performing **large data updates** (e.g., refreshing weather history).
- Running **nightly or weekly scheduled jobs**.
- Sending **bulk notifications or emails**
- Helps handle **large data volumes efficiently**
- Provides **flexibility** in executing jobs.

Phase 6: User Interface Development

1.Lightning App Builder

Purpose

- **Lightning App Builder** is a **point-and-click tool in Salesforce** that allows admins and developers to create and customize pages without writing any code.
- It supports Home Pages, Record Pages, and App Pages, enabling users to tailor the interface to business needs.
- With drag-and-drop components, it reduces dependency on developers and accelerates deployment of user-friendly pages.

Key Features

1. Drag-and-Drop Interface

- Allows placing components (standard, custom, or LWC) anywhere on the page.
- Users can preview the layout in real time.

2. Multiple Page Types

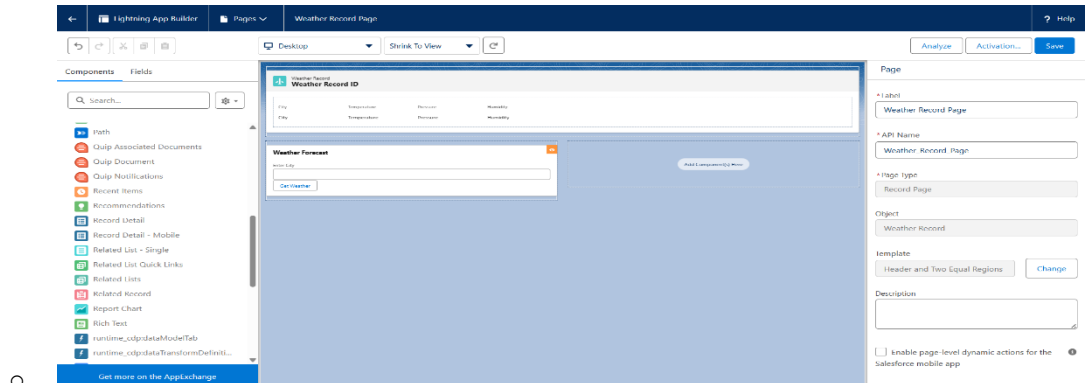
- **Home Page:** Displays key information on login, dashboards, or quick actions.
- **Record Page:** Customizes the layout of object records, showing fields, related lists, and LWCs.
- **App Page:** Used for custom applications like a Weather Dashboard with multiple components.

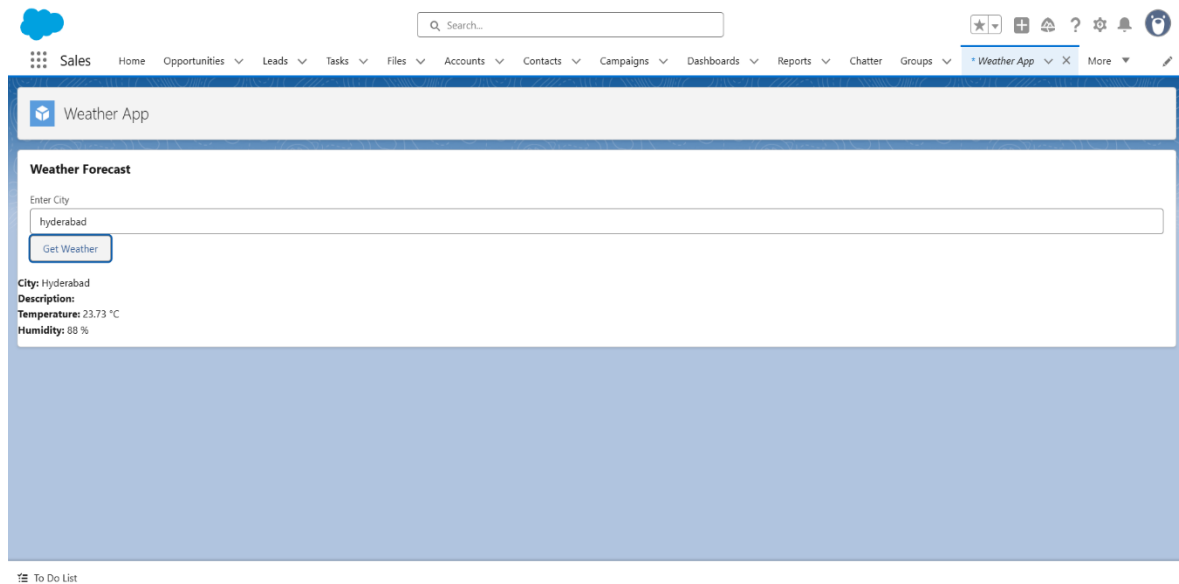
3. Responsive Layouts

- Choose from 1-column, 2-column, or 3-column layouts.
- Adjust component width and placement for desktop and mobile devices.

4. Component Types Supported

- **Standard Components:** Reports, Tasks, Chatter feeds, Related Lists.
- **Custom Components:** Created via Lightning Web Components or Aura Components.
- **Dynamic Components:** Components that display based on filters, user roles, or record data.





2. Record Pages

Purpose

- **Record Pages** in Salesforce allow you to **customize the layout of individual records** of any object, whether standard (Account, Contact) or custom (Weather_Record__c).
- They control **which fields, components, and related lists** appear for different users, profiles, and record types.
- Improves **data visibility and user experience** by displaying only relevant information.

Key Features

1. Custom Layouts by Object & Record Type

- Different layouts for different profiles or record types.
- Example: Weather_Record__c might show additional fields for “Admin” users but fewer fields for standard users.

2. Component-Based Design

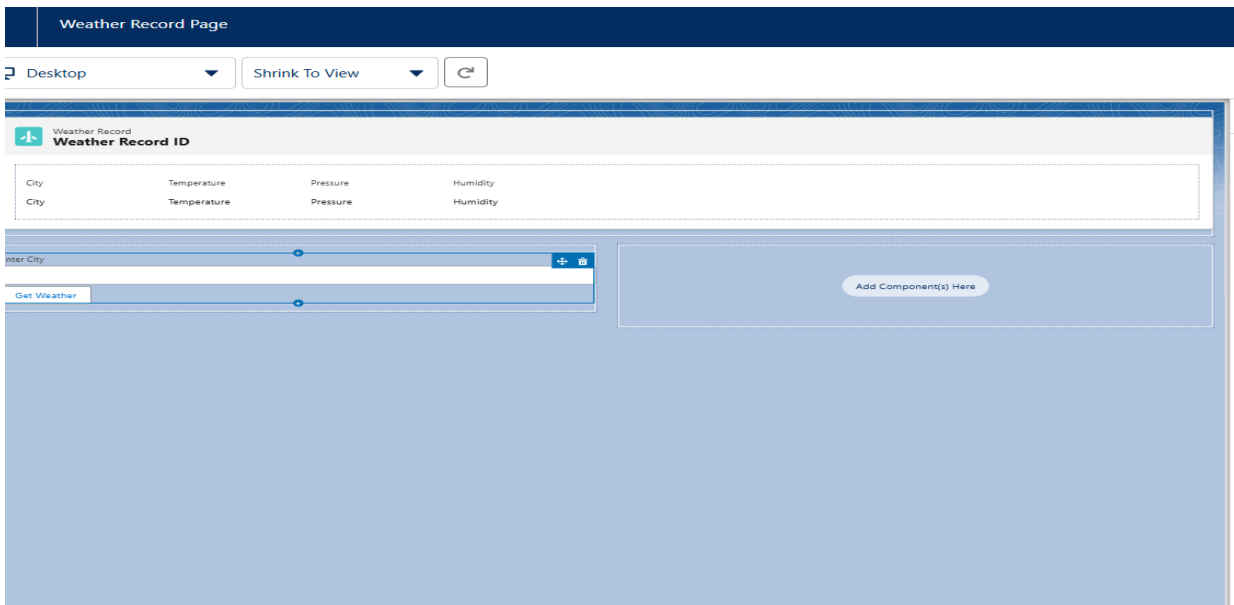
- Use **Lightning App Builder** to drag components like:
 - Standard components: Related lists, Highlights Panel, Tabs.
 - Custom components: WeatherApp LWC, charts, or dynamic dashboards.

3. Dynamic Visibility Rules

- Components can appear or hide based on:
 - Field values (e.g., Status__c = “Confirmed”)
 - User profile or role
 - Record type

4. Mobile & Desktop Optimization

- Record Pages can be designed to work seamlessly on mobile Salesforce app.



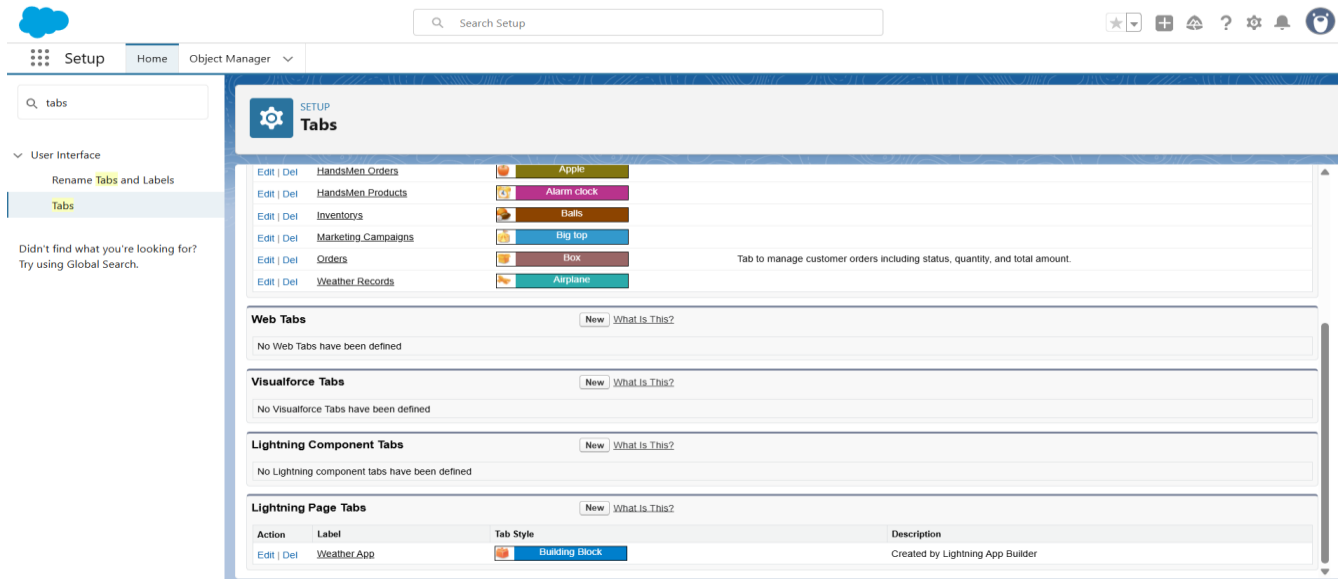
3. Tabs

Purpose

- **Tabs** allow users to **navigate between different objects, apps, or Lightning Pages** efficiently.
- They help **organize Salesforce UI** and provide quick access to key data.

Key Features

1. **Custom Object Tabs**
 - Link to custom objects like **Weather Records, Cities**, etc.
 - Appear in App navigation bar.
2. **Lightning Page Tabs**
 - Link directly to Lightning Pages (e.g., **Weather App Page**).
 - Can be used in App navigation or Utility Bar.
3. **Web Tabs**
 - Embed external web content or dashboards inside Salesforce.
4. **Visualforce & Lightning Component Tabs**
 - Display Visualforce pages or standalone Lightning Components as tabs.
5. **App-Specific Tabs**
 - Tabs can be added to **specific apps** via **App Manager → Edit → Navigation Items**.
 - Order of tabs can be customized for usability.



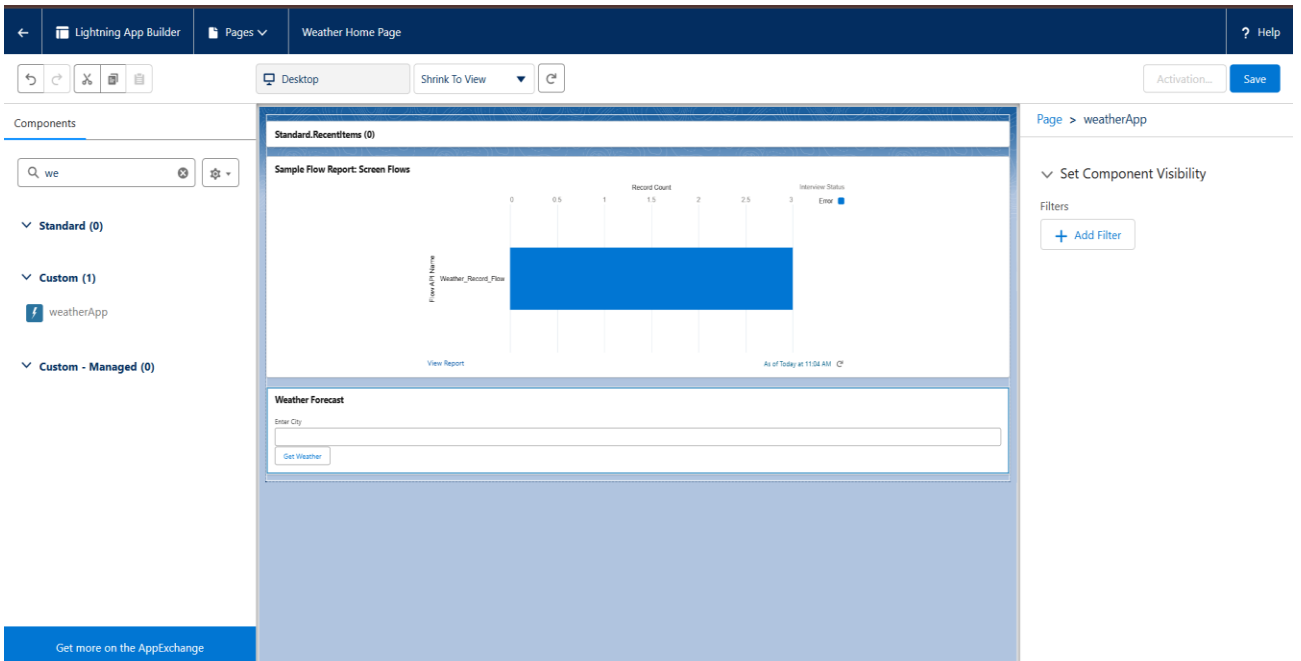
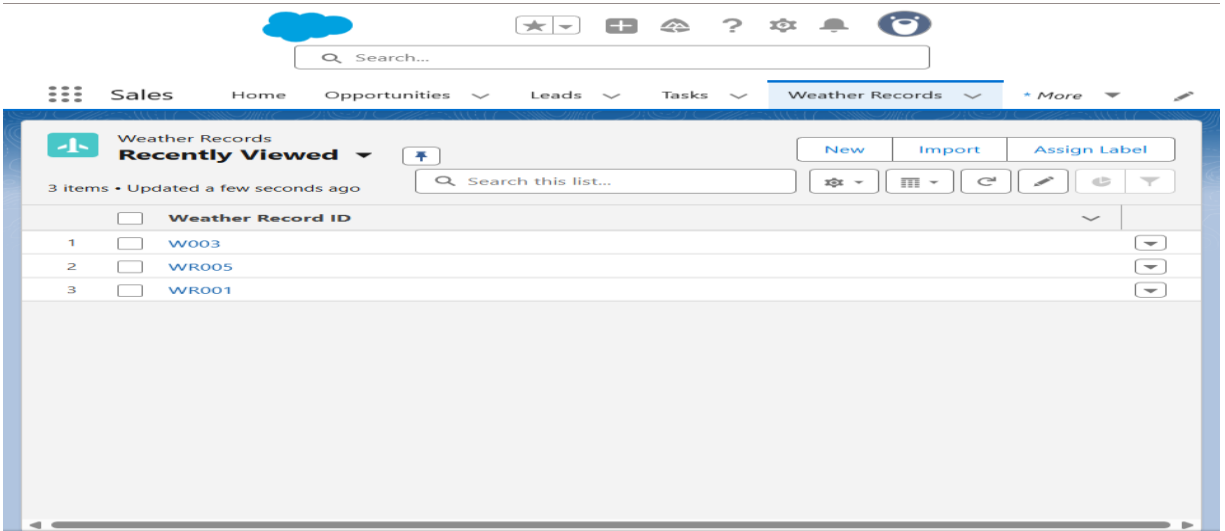
4.Home Page Layouts

Overview

Home Page Layouts control **what users see immediately after logging into Salesforce**. They allow organizations to highlight key information, reports, tasks, or custom components like the Weather App for fast access. Properly designed layouts improve **user productivity and data visibility**.

Key Features

1. **Custom Layouts**
 - Choose from 1-column, 2-column, or 3-column templates.
 - Adjust components for desktop and mobile users.
2. **Drag-and-Drop Components**
 - Standard Components: Reports, Tasks, News, Recent Items.
 - Custom Components: Lightning Web Components like WeatherApp, charts, dashboards.
3. **Dynamic Visibility**
 - Show or hide components based on:
 - Profile
 - Role
 - Record type or field values
4. **Quick Access to Information**
 - Displays real-time or summarized data for immediate decision-making.
 - Reduces the need for users to navigate multiple pages.



5. Utility Bar

Overview

The **Utility Bar** is a persistent toolbar at the bottom of Salesforce apps. It provides quick access to essential tools and components, improving workflow efficiency without leaving the current page.

Key Features

1. **Persistent Access**
 - Available on all pages in the app.
 - Users can open utilities like calculators, WeatherApp, Tasks, or Notes while navigating other pages.
2. **Custom Components**
 - Drag Lightning Web Components or standard components into the Utility Bar.
 - Example: WeatherApp can show live weather in a small, collapsible panel.
3. **Configurable Properties**
 - Label, icon, panel size (small, medium, large).
 - Visibility rules based on profiles or roles.
 - Auto-collapse or dock options for better UI.
4. **Enhances Productivity**
 - Provides tools at users' fingertips without navigating away.
 - Supports multitasking, e.g., viewing weather while editing records.

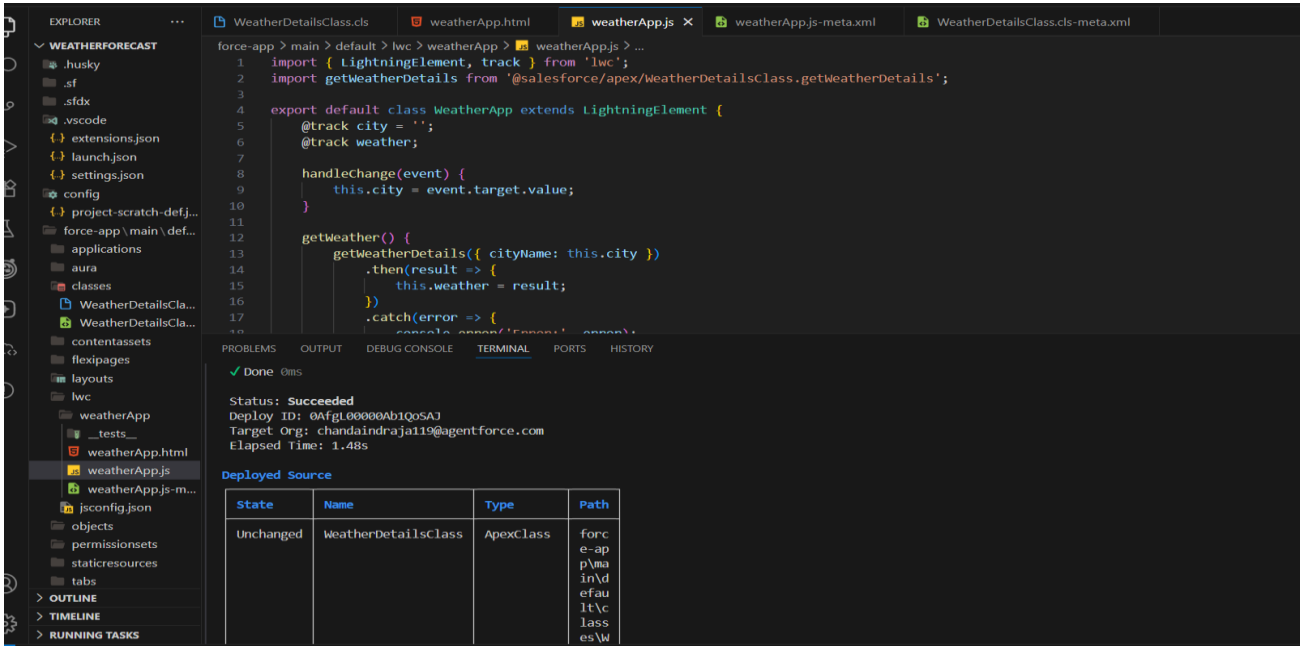
6. Lightning Web Components (LWC)

Overview

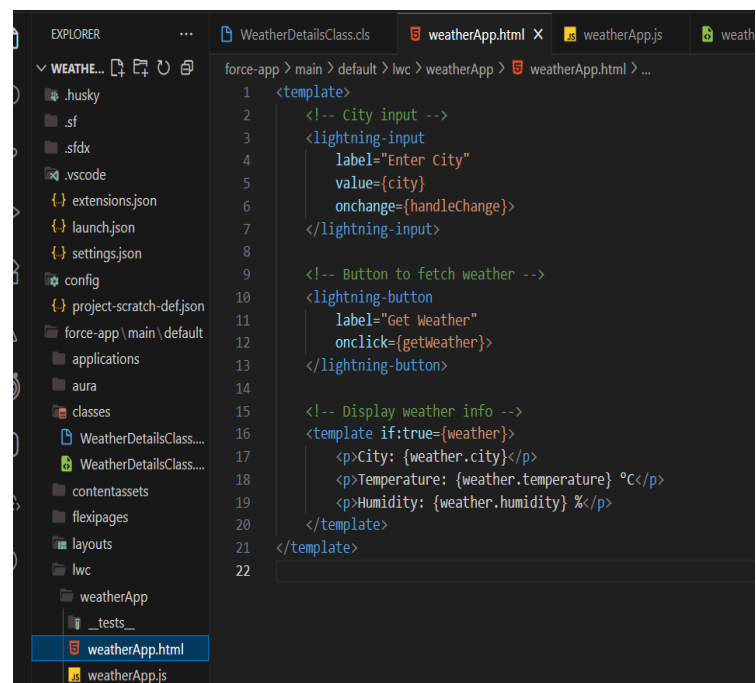
- Lightning Web Components (LWC) is **Salesforce's modern UI framework** for building fast, reusable, and interactive components.
- Based on **modern web standards** (HTML, JavaScript, CSS).
- LWCs can run on **Salesforce desktop and mobile**, integrate with **Apex**, and interact with other components.

Key Features

1. **Component-Based Architecture**
 - Modular, reusable, and encapsulated.
 - Each LWC consists of **HTML, JS, CSS, and meta XML**.
2. **Reactive Data Binding**
 - Uses **@track** and **@api** decorators to bind data between JS and HTML.
3. **Integration with Salesforce Data**
 - Can call **Apex methods** to fetch or update records.
 - Can use **Wire Adapters** for reactive Salesforce data.
4. **Performance & Efficiency**
 - Runs client-side with minimal server requests.
 - Lightweight and optimized for Salesforce Lightning Experience.



```
1 import { LightningElement, track } from 'lwc';
2 import getWeatherDetails from '@salesforce/apex/WeatherDetailsClass.getWeatherDetails';
3
4 export default class WeatherApp extends LightningElement {
5   @track city = '';
6   @track weather;
7
8   handleChange(event) {
9     this.city = event.target.value;
10   }
11
12   getWeather() {
13     getWeatherDetails({ cityName: this.city })
14       .then(result => {
15         this.weather = result;
16       })
17       .catch(error => {
18         console.error('Error:', error);
19       });
20   }
21 }
22
```



7. Apex with LWC

Overview

- LWCs can communicate with **server-side Apex** to handle complex logic or access Salesforce data.
- Apex methods are called using **@AuraEnabled** decorators.

Key Features

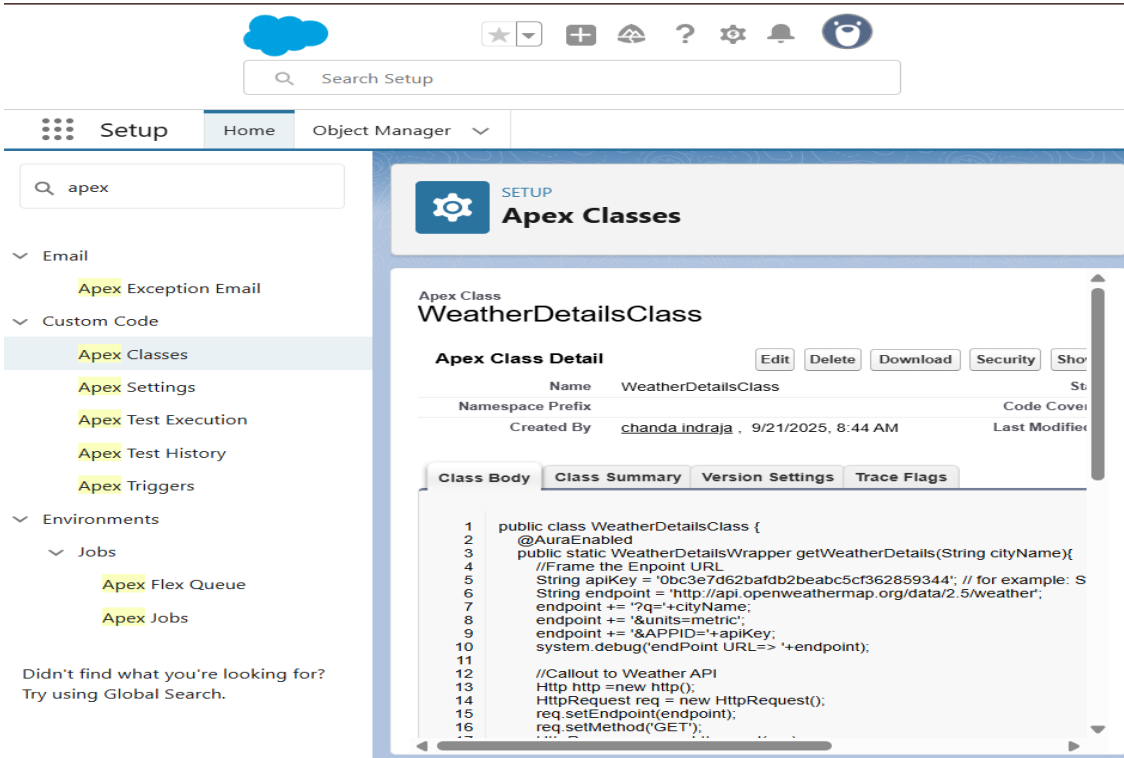
1. **Imperative Calls**
 - JavaScript calls Apex on-demand (e.g., after user clicks “Get Weather”).
2. **Reactive Calls with @wire**
 - Automatically fetches Salesforce data and updates the UI when records change.
3. **Data Processing**
 - Apex handles filtering, calculations, or API integration, then returns data to LWC.

Step-by-Step Implementation

1. **Create Apex Class**
 - Example: WeatherDetailsClass with @AuraEnabled(cacheable=true) method.
 - Handles fetching weather details from API or Salesforce records.
2. **Call Apex from LWC**
 - **Imperatively** in JS:

```
getWeatherDetails({ cityName: this.city })  
.then(result => { this.weather = result; })  
.catch(error => { console.error(error); });
```

- **Reactively** with @wire decorator for automatic updates.
3. **Handle Success & Errors**
 - Update component state on success.
 - Use catch() for errors or display **toast messages**.
 4. **Deploy Apex Class and LWC**
 - Deploy both together to Salesforce.
 5. **Test Component**
 - Place LWC on a Lightning Page → verify Apex data is fetched and displayed.



8. Events in LWC

Overview

- **Events in LWC** allow components to communicate **within the component hierarchy**.
- Two main types:
 1. **Standard DOM Events** – e.g., click, change.
 2. **Custom Events** – e.g., sending weather data from a child component to a parent component.

Key Features

1. **CustomEvent**
 - Send structured data using detail object.
 - Example: { detail: { city: 'Mumbai', temperature: 25 } }.
2. **Event Bubbling**
 - Events propagate up the DOM to parent components.
3. **Component Interaction**
 - Enables reusable child components that inform parent components about user actions.

. 9.Wire Adapters

Overview

- **Wire Adapters** in LWC provide a **reactive connection** between Salesforce data and your Lightning Web Component.
- Data fetched via wire adapters automatically **updates the component when underlying Salesforce data changes**.
- Ideal for **read-only or reactive UI elements**, such as displaying Weather Records in real-time.

Key Features

1. **Reactive Data Binding**
 - Automatically updates the component when data changes in Salesforce.
2. **Integration with Apex or Standard Objects**
 - Use `@wire` to call **Apex methods** or **standard Lightning Data Service adapters** like `getRecord` or `getListUi`.
3. **Declarative Syntax**
 - Simplifies code; no need to explicitly handle server requests or callbacks.

10. Imperative Apex Calls – Detailed Documentation

Overview

- **Imperative Apex Calls** in LWC are used to **call Apex methods on-demand**, typically triggered by user actions like button clicks.
- Unlike `@wire`, which is reactive, imperative calls are **explicit and controlled by the component**.

Key Features

1. **On-Demand Execution**
 - Fetch or process data when required, e.g., after clicking “Get Weather”.
2. **Supports Promises**
 - Uses `.then()` and `.catch()` to handle success and errors.
3. **Flexibility**
 - Suitable for dynamic operations that depend on user input.

Phase 7: Integration & External Access

1. Named Credentials

Purpose

- Securely store authentication information for external APIs.
- Simplifies API callouts by avoiding hardcoding credentials in Apex code.
- Manages authentication protocols such as OAuth 2.0, Password Authentication, or Named Principal.
- Enhances security by controlling access centrally.

Key Components

- **Label & Name:** User-friendly label and unique API name for reference in Apex.
- **URL:** Base URL of the external service.
- **Identity Type:** Defines who is making the callout:
 - Named Principal – All users share the same authentication.
 - Per User – Each Salesforce user uses their own credentials.
- **Authentication Protocol:** Type of authentication used for the API:
 - Password Authentication
 - OAuth 2.0
 - AWS Signature
 - JWT Bearer Token
- **External Credential (Optional):** Can link to external credentials stored securely.
- **Callout Enabled:** Must be checked to allow callouts using this credential.
- **Usage in Apex:**

```
HttpRequest req = new HttpRequest();  
req.setEndpoint('callout:WeatherAPI_NC/v1/current.json?q=London');  
req.setMethod('GET');  
HttpResponse res = new Http().send(req);  
System.debug(res.getBody());
```

The screenshot displays the Salesforce Setup interface. On the left, the 'Setup' menu is visible with 'Security' expanded and 'Named Credentials' selected. The main content area shows the configuration for a Named Credential named 'WeatherAPI_NC'. The configuration includes:

- Label:** WeatherAPI_NC
- Name:** WeatherAPI_NC
- URL:** https://api.weatherapi.com
- Enabled for Callouts:** A toggle switch that is currently turned on.
- Authentication:**
 - External Credential:** WeatherAPI_EC
 - Client Certificate:** (Field is empty)
- Callout Options:**
 - Generate Authorization Header:** A checkbox that is checked.
 - Allow Formulas in HTTP Header:** (Field is empty)

Buttons for 'Edit' and 'Delete' are located at the top right of the configuration panel.

2. External Services

Purpose

- Connect Salesforce to external APIs **declaratively**.
- Automatically generate **actions for Flows**.
- Simplifies integration for admins and non-developers.
- Supports both **REST and SOAP APIs**.

Key Features

- Declarative API integration with no Apex required.
- Uses **Swagger/OpenAPI schema** to define API structure.
- Actions are **automatically created** in Salesforce.
- Works with **Named Credentials** for secure authentication.
- Can be used in **Screen Flows, Scheduled Flows, and Record-Triggered Flows**.
- Supports reusability and governance of API calls.

Steps:

- Setup → External Services → New External Service
- Provide Name, Description, Service Schema (Swagger/OpenAPI URL)
- Select Named Credential
- Salesforce auto-generates actions usable in Flow

The screenshot shows the Salesforce Setup interface. The left sidebar has a search bar with "named" entered, and the "Named Credentials" link is highlighted under the "Security" section. The main content area displays the configuration for the "WeatherAPI_EC" Named Credential. The configuration includes a label "WeatherAPI_EC", a name "WeatherAPI_EC", and an authentication protocol of "Custom". Below this is a "Managed Package Access" section. The "Related Named Credentials" section shows a table with one entry: "WeatherAPI_NC" with a URL of "https://api.weatherapi.com". The "Principals" section shows a table with one entry: "WeatherAPI_Principal" with an authentication type of "Unknown".

Label	Name
WeatherAPI_EC	WeatherAPI_EC

Authentication Protocol: Custom

Managed Package Access

Created By Namespace ⓘ

Label	Name	URL
WeatherAPI_NC	WeatherAPI_NC	https://api.weatherapi.com

S...	Parameter Name	A...	Aut...	Actions
1	WeatherAPI_Principal	1	Unknown	⌵

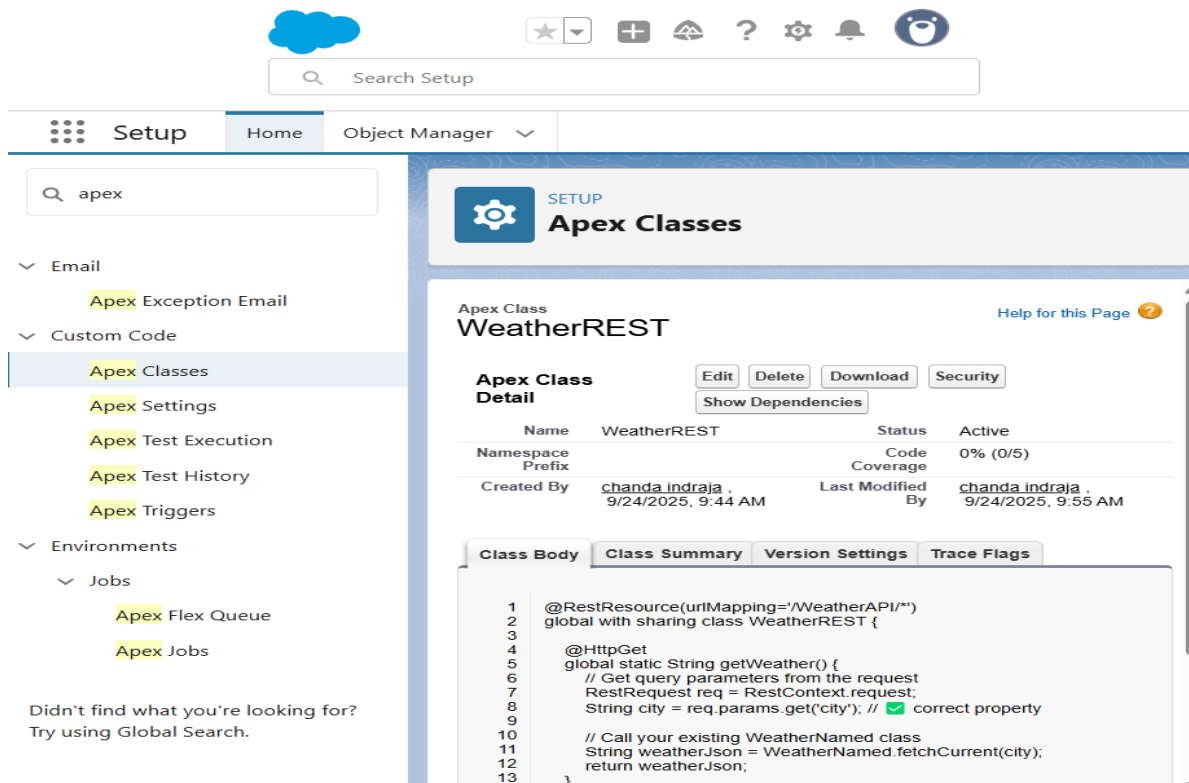
3. Web Services (REST/SOAP)

Purpose

- Expose Salesforce data to external systems.
- Consume external web services within Salesforce.
- Enable integration with external applications using standard protocols.
- Supports both **REST (lightweight)** and **SOAP (structured)** protocols.

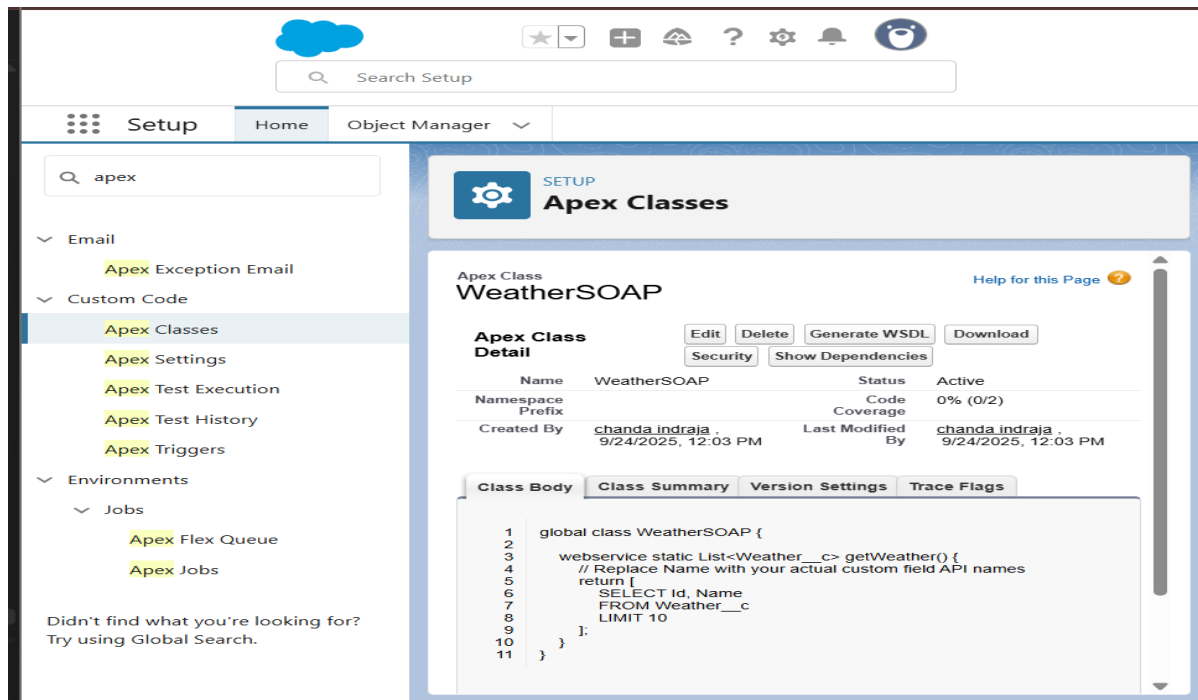
Key Features

- **REST Services**
 - Lightweight, stateless, uses HTTP methods (GET, POST, PUT, DELETE).
 - Supports JSON and XML formats.
 - Easier to use for modern web and mobile apps.
- **SOAP Services**
 - Standardized protocol using XML.
 - Strongly typed operations, supports WSDL.
 - Preferred for enterprise-level integrations.



The screenshot displays the Salesforce Setup interface. The left sidebar shows the navigation menu with 'Setup' selected. Under 'Custom Code', 'Apex Classes' is highlighted. The main content area is titled 'Apex Classes' and shows details for the 'WeatherREST' class. The class is active, created by 'chanda.indraja' on 9/24/2025 at 9:44 AM, and has 0% code coverage. The 'Class Body' tab is selected, showing the following code:

```
1 @RestResource(urlMapping='/WeatherAPI/*')
2 global with sharing class WeatherREST {
3
4     @HttpGet
5     global static String getWeather() {
6         // Get query parameters from the request
7         RestRequest req = RestContext.request;
8         String city = req.params.get('city'); // correct property
9
10        // Call your existing WeatherNamed class
11        String weatherJson = WeatherNamed.fetchCurrent(city);
12        return weatherJson;
13    }
14 }
```



4. Callouts

Purpose

- Enable Salesforce to **communicate with external APIs** or web services.
- Retrieve or send data from/to external systems.
- Used in integrations, third-party services, or external data synchronization.

Key Features

- Supports **HTTP methods**: GET, POST, PUT, DELETE.
- Can use **Named Credentials** for secure authentication.
- Can also use **Remote Site Settings** for older setups.
- Supports **synchronous** (immediate response) and **asynchronous** (future methods, batch Apex) callouts.
- Can handle **REST and SOAP APIs**.
- Use **Named Credentials** instead of Remote Site Settings for security.
- Always handle exceptions with `try-catch`.

5. Platform Events in Salesforce

Purpose

- Enable **event-driven architecture** inside Salesforce.
- Send and receive custom notifications between Salesforce processes or external systems.
- Useful for real-time updates and decoupled integrations.

Key Features

- **Custom Event Objects** with fields like any other object.
- Events are **published** and **subscribed** asynchronously.
- Supports **Publish-Subscribe model**:
 - **Publisher**: Creates and publishes events.
 - **Subscriber**: Triggers, Flows, or external systems that react to events.
- Can be used in **Apex, Flows, and External Systems (Streaming API)**.
- Durable events: Can retain events for up to **24 hours**.

The screenshot shows the Salesforce Setup interface for Platform Events. The left sidebar contains a navigation menu with options like MuleSoft, Anypoint Platform Setup, Einstein, Einstein Platform, Einstein Bots, Einstein AI, Custom Code, Platform Cache, Integrations, Platform Events (selected), Security, Platform Encryption, Encryption Settings, and Key Management. The main content area is titled 'Platform Events' and includes a search bar, a table of event objects, and sections for Standard Fields, Custom Fields & Relationships, and Triggers.

Singular Label	Plural Label	Object Name	API Name	Event Type	Publish Behavior	Created By	Modified By
WeatherUpdateEvent	WeatherUpdateEvents	WeatherUpdateEvent	WeatherUpdateEvent__e	High Volume	Publish Immediately	chanda.indraja	chanda.indraja

Action	Field Label	Field Name	Data Type	Controlling Field	Indexed
	Created By	CreatedBy	Lookup(User)		
	Created Date	CreatedDate	Date/Time		
	Event UUID	EventUuid	Text(36)		
	Replay ID	ReplayId	External Lookup		

Action	Field Label	API Name	Data Type	Indexed	Controlling Field	Modified By
Edit / Del	City	City__c	Text(3)			chanda.indraja 9/24/2025, 10:39 AM
Edit / Del	Condition	Condition__c	Text(18)			chanda.indraja 9/24/2025, 10:40 AM
Edit / Del	Temperature	Temperature__c	Number(18, 0)			chanda.indraja 9/24/2025, 10:39 AM

The screenshot shows the Salesforce Execution Log window. It displays a log line with the timestamp 23:11:50:044, the event type USER_DEBUG, and the details [8][DEBUG]Event published: true. A dialog box titled 'Log line #35 details' is open, showing the same log line details.

Timestamp	Event	Details
23:11:50:044	USER_DEBUG	[8][DEBUG]Event published: true

Log line #35 details

23:11:50:044 USER_DEBUG [8][DEBUG]Event published: true

OK

6. Change Data Capture (CDC)

Purpose

- Track **create, update, delete, and undelete events** on Salesforce records in real time.
- Enable **asynchronous integration** with internal or external systems.
- Helps in **synchronizing Salesforce data** with other systems.
- Works with **any standard or custom object**, including `Weather__c`.

Key Features

- Real-time tracking of **record changes**.
- Automatically generates **Change Event objects** for tracked objects.
- Supports **Apex Triggers, Flows, and External Subscribers**.
- Captures **all field changes** automatically.
- Can be integrated with **Streaming API** or external systems.
- Works with **standard objects** and **custom objects**.

The screenshot shows the Salesforce Setup interface. On the left is a navigation sidebar with a search bar containing 'apex'. Below the search bar, there are expandable sections: 'Email' (with 'Apex Exception Email'), 'Custom Code' (with 'Apex Classes', 'Apex Settings', 'Apex Test Execution', 'Apex Test History', and 'Apex Triggers' which is highlighted), 'Environments' (with 'Jobs' and 'Apex Flex Queue', 'Apex Jobs'), and a message 'Didn't find what you're looking for? Try using Global Search.' The main content area is titled 'SETUP Apex Triggers'. It features a search bar, a 'Help for this Page' link, and a section for 'Apex Trigger WeatherChangeTrigger'. This section includes buttons for 'Edit', 'Delete', 'Download', and 'Show Dependencies'. Below these buttons is a table with details: Name (WeatherChangeTrigger), sObject Type, Code Coverage (0% (0/7)), Status (Active), Created By (chanda.indraja, 9/24/2025, 10:46 AM), Last Modified By (chanda.indraja, 9/24 11:01 AM), and Namespace Prefix. At the bottom, there are tabs for 'Apex Trigger', 'Version Settings', and 'Trace Flags'. The 'Apex Trigger' tab is active, showing the trigger code:

```
1 trigger WeatherChangeTrigger on Weather__ChangeEvent (after insert) {
2   for (Weather__ChangeEvent event : Trigger.New) {
3
4     // Access changed record IDs
5     List<Id> changedRecordIds = event.ChangeEventHeader.getRecordIds();
6     System.debug('Record changed: ' + changedRecordIds);
7
8     // Access your custom fields
9     System.debug('City: ' + event.get('City__c'));
10    System.debug('Temperature: ' + event.get('Temperature__c'));
11    System.debug('Condition: ' + event.get('Condition__c'));
12
13    // Optional: publish Platform Event
14    Weather__IndateEvent e = new Weather__IndateEvent(e.get('City__c'), e.get('Temperature__c'), e.get('Condition__c'));
```

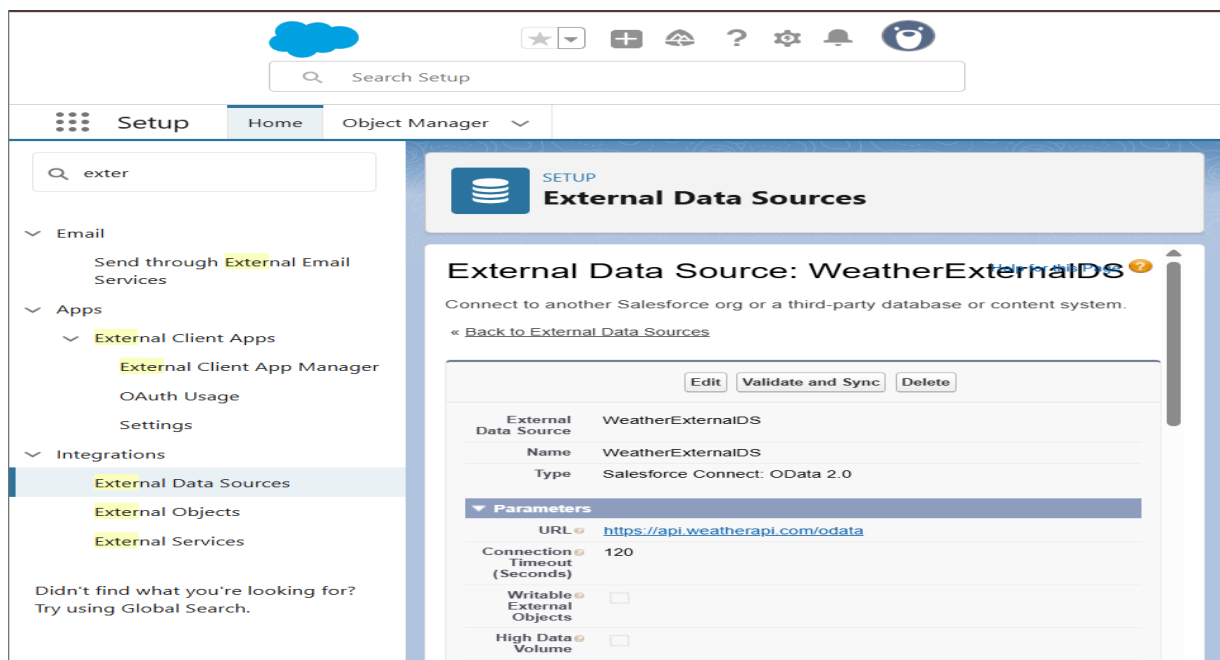
7. Salesforce Connect

Purpose

- Integrate external data sources directly into Salesforce without storing the data in Salesforce.
- Enables **real-time access** to external objects via **OData, REST, or custom adapters**.
- Ideal for **large datasets** or external systems where replication is not practical.

Key Features

- Supports **OData 2.0 and 4.0** protocols.
- Creates **External Objects** in Salesforce representing external tables.
- Provides **read, create, update, and delete** access depending on external system capabilities.
- Works with **Lightning, Visualforce, and Apex**.
- Can combine **external objects with standard Salesforce objects** using **relationship**



8. API Limits

Purpose

- Salesforce enforces **limits on API calls** to protect system resources.
- Helps track and manage usage for integrations and automation.

Key Points

- **API Request Limits** vary by **edition, license type, and user**.

- Types of API calls:
 - REST API
 - SOAP API
 - Bulk API
 - Streaming API
 - Tooling API
- Monitor usage via: **Setup** → **System Overview** → **API Usage**, or **REST Limits resource**.
- **Exceeded limits** return `REQUEST_LIMIT_EXCEEDED` errors.

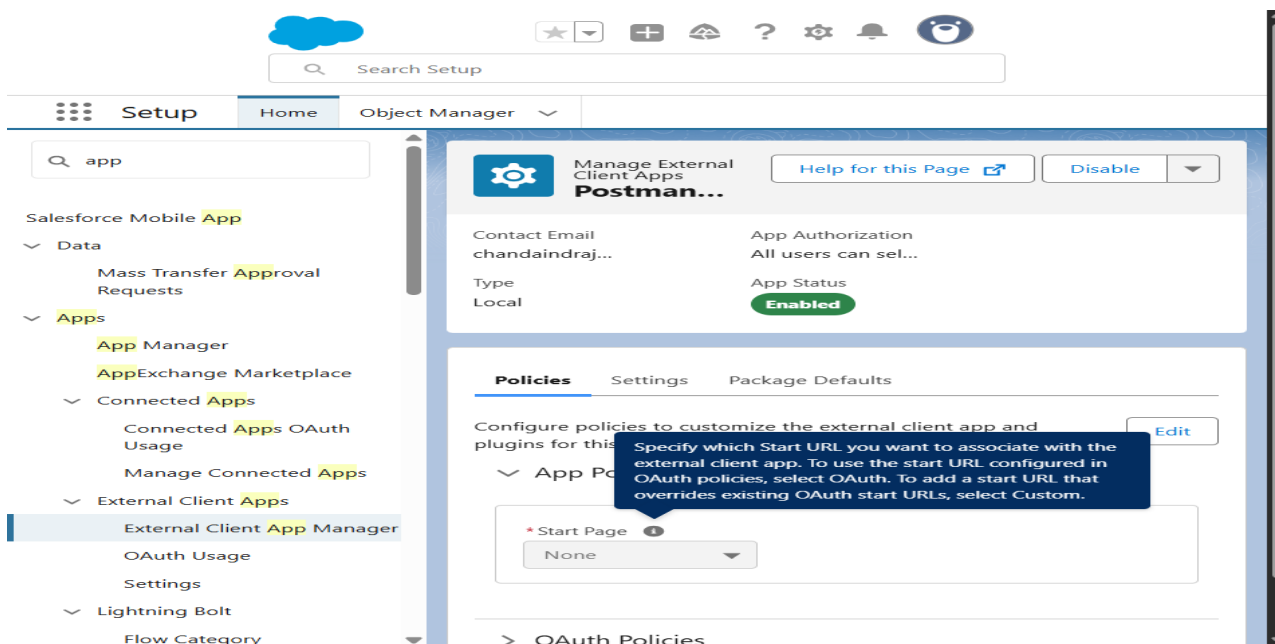
9. OAuth & Authentication

Purpose

- Secure integration with Salesforce or external systems.
- Provides **token-based authentication** without storing passwords.
- Supports **single sign-on (SSO)** and **API access**.

Key Features

- Supports **OAuth 2.0 flows**:
 - Authorization Code
 - Username-Password
 - JWT Bearer
 - Refresh Token
- Provides **access token** for API calls.
- Tokens can expire; refresh tokens allow re-authentication without user intervention.



10.Remote Site Settings

Purpose

- Allows Salesforce to **perform HTTP or HTTPS callouts** to external systems.
- Required when using **Apex HTTP callouts** to URLs that are **not secured via Named Credentials**.
- Ensures **security by allowing only trusted URLs** to be accessed from Salesforce.

Key Features

- Lets Salesforce **call external web services or APIs**.
- Provides a **centralized place** to manage external URLs.
- Required for **REST/SOAP API integration**, webhooks, or external system data fetch.
- Works with **all Apex HTTP callouts**, including GET, POST, PUT, DELETE.
- Supports **HTTPS** endpoints for secure communication.
- Can be used in combination with **Named Credentials** for advanced authentication.

The screenshot displays the Salesforce user interface for the 'Remote Site Settings' page. The top navigation bar includes the Salesforce logo, a search bar labeled 'Search Setup', and several utility icons. The left sidebar shows the 'Setup' menu with 'Home' and 'Object Manager' tabs. Under the 'Security' section, 'Remote Site Settings' is highlighted. The main content area is titled 'Remote Site Settings' and contains a 'Remote Site Details' section. This section shows a table with one entry: 'WeatherAPI_RS'. The table includes fields for 'Remote Site Name', 'Remote Site URL' (https://api.weatherapi.com), 'Disable Protocol Security' (unchecked), 'Description' (Active), and 'Created By' (chanda.indraja, 9/24/2025, 11:15 AM). Action buttons 'Edit', 'Delete', and 'Clone' are available for this entry. A 'Help for this Page' link is also present.

Remote Site Detail		Edit	Delete	Clone
Remote Site Name	WeatherAPI_RS	Modified By chanda.indraja, 9/24/2025, 11:15 AM		
Remote Site URL	https://api.weatherapi.com			
Disable Protocol Security	☐			
Description	Active			
Created By	chanda.indraja, 9/24/2025, 11:15 AM	Edit	Delete	Clone

Phase 8: Data Management & Deployment

Effective data management and deployment strategies ensure that Salesforce data is clean, secure, and easy to move between environments (like sandbox and production).

1. Data Import Wizard

- **Purpose:** Easy-to-use tool for importing data (CSV files) directly into Salesforce.
- **Usage:** Available inside Salesforce Setup → Data Import Wizard.
- **Features:**
 - Import up to **50,000 records**.
 - Supports standard objects (Accounts, Contacts, Leads, Campaign Members) and custom objects.
 - Provides **field mapping** interface.
 - Can update existing records or insert new ones.
- **Best Use Case:** Small data loads by admins (no external tool needed).

The screenshot shows the Salesforce Data Import Wizard interface. At the top, there's a navigation bar with 'Setup', 'Home', and 'Object Manager'. Below this, the 'Data Import Wizard' title is displayed. A table titled 'Recent Import Jobs' shows a single job with status 'Closed', object 'City', 4 records created, 0 updated, 0 failed, and a processing time of 163 ms. Below the table is a 'Bulk Api Monitoring' button. A large light blue banner contains the text 'Before you import your data...' and lists three key points: 'Clean up your data import file', 'Make sure your field names match Salesforce field names', and 'Don't import too many records at once'. The banner also includes links to 'Watch video' and 'View a list of Salesforce data fields'.

Status	Object	Records Created	Records Updated	Records Failed	Start Date	Processing Time (ms)
Closed	City	4	0	0	09-25-2025 09:40	163

Bulk Api Monitoring

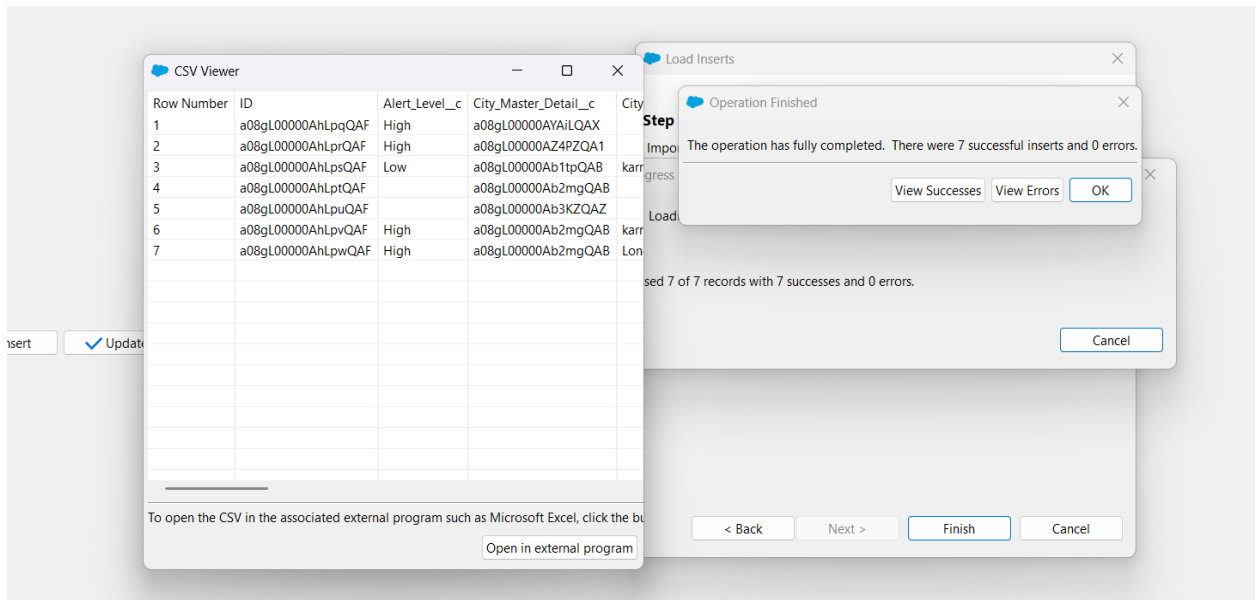
Before you import your data...

- Clean up your data import file**
You'll have fewer errors to resolve if your data file is clean and free of duplicates. [Watch video](#)
- Make sure your field names match Salesforce field names**
You'll be required to map your data fields to Salesforce data fields. Data in unmapped fields is not imported. [View a list of Salesforce data fields.](#)
- Don't import too many records at once**
Using the Data Import Wizard, import up to 50,000 records at a time. Importing too many records can slow down your org for all users, especially during periods of peak usage.

2. Data Loader

- **Purpose:** Advanced client application for **bulk import/export** of data.
- **Features:**
 - Handles **millions of records**.
 - Supports Insert, Update, Upsert, Delete, Hard Delete, Export.

- Requires CSV files.
- Available as **UI** and **Command-line**.
- **Best Use Case:** Large data migrations, scheduled automated data jobs.



Row Number	ID	Alert_Level_c	City_Master_Detail_c	City_Name_c	City_c	CreatedById	CreatedDate	Humidity_c	Id	IsDeleted	LastActivityDate	LastModifiedById	LastModifiedD
1	a08gL00000AhLpqQAF	High	a08gL00000AYAILQAX		a08gL00000AYAVRQA5	005gL000004IGIQAM	2025-09-22T10:38:41.000Z	60.0	a08gL00000AhLpqQAF	false		005gL000004IGIQAM	2025-09-23T11:
2	a08gL00000AhLprQAF	High	a08gL00000AZ4PZQA1			005gL000004IGIQAM	2025-09-22T18:20:15.000Z		a08gL00000AhLprQAF	true		005gL000004IGIQAM	2025-09-23T11:
3	a08gL00000AhLpsQAF	Low	a08gL00000Ab1tpQAB	karnataka	a08gL00000Ab1tpQAB	005gL000004IGIQAM	2025-09-23T10:53:54.000Z	70.0	a08gL00000AhLpsQAF	false		005gL000004IGIQAM	2025-09-23T11:
4	a08gL00000AhLptQAF		a08gL00000Ab2mgQAB			005gL000004IGIQAM	2025-09-23T11:02:24.000Z	70.0	a08gL00000AhLptQAF	true		005gL000004IGIQAM	2025-09-23T11:
5	a08gL00000AhLpuQAF		a08gL00000Ab3KZQAZ			005gL000004IGIQAM	2025-09-23T11:13:38.000Z	70.0	a08gL00000AhLpuQAF	true		005gL000004IGIQAM	2025-09-23T11:
6	a08gL00000AhLpvQAF	High	a08gL00000Ab2mgQAB	karnataka	a08gL00000Ab3KZQAZ	005gL000004IGIQAM	2025-09-23T17:33:50.000Z	90.0	a08gL00000AhLpvQAF	false		005gL000004IGIQAM	2025-09-23T11:
7	a08gL00000AhLpwQAF	High	a08gL00000Ab2mgQAB	London	a08gL00000Ab3KZQAZ	005gL000004IGIQAM	2025-09-24T17:54:53.000Z	54.0	a08gL00000AhLpwQAF	false		005gL000004IGIQAM	2025-09-24T11:

3. Duplicate Rules

- **Purpose:** Prevent or manage duplicate records in Salesforce.
- **Setup:** Setup → Duplicate Rules.
- **Features:**
 - Compare new/existing records to find duplicates.
 - Supports **blocking** or **allowing but alerting** duplicates.

- Works with **Matching Rules** (e.g., match by Email, Phone, Name).
- **Best Use Case:** Keep clean data in Leads, Accounts, Contacts.



★

+

🏠

?

⚙️

🔔



Q

Search Setup

⋮

Setup

Home

Object Manager

Q

duplicate

▼ Data

▼ Duplicate Management

Duplicate Error Logs

Duplicate Rules

Matching Rules

Didn't find what you're looking for?
Try using Global Search.

d

SETUP

Duplicate Rules

Weather Duplicate Rule

Help for this Page

Duplicate Rule Detail

EditDeleteCloneActivate

Rule Name	Weather Duplicate Rule	Order	1 of 1 [Reorder]
Description	Prevents duplicate Weather records based on Observation Time and City.		
Object	Weather		
Record-Level Security	Enforce sharing rules		
Action On Create	Allow	Operations On Create	☒ Alert ☒ Report
Action On Edit	Allow	Operations On Edit	☒ Alert ☐ Report
Alert Text	Use one of these records?		
Active	☐		
Conditions	(Weather: City EQUALS Mumbai) AND (Weather: City EQUALS karnataka)		
Created By	chanda indraja, 9/25/2025, 3:10 AM	Modified By	chanda indraja, 9/25/2025, 3:10 AM

EditDeleteCloneActivate



★

+

🏠

?

⚙️

🔔



Q

Search Setup

⋮

Setup

Home

Object Manager

Q

dupl

▼ Data

▼ Duplicate Management

Duplicate Error Logs

Duplicate Rules

Matching Rules

Didn't find what you're looking for?
Try using Global Search.

d

SETUP

Duplicate Rules

Weather Duplicate Rule

Help for this Page

Duplicate Rule Detail

EditDeleteCloneDeactivate

Rule Name	Weather Duplicate Rule	Order	1 of 1 [Reorder]
Description	Prevents duplicate Weather records based on Observation Time and City.		
Object	Weather		
Record-Level Security	Enforce sharing rules		
Action On Create	Allow	Operations On Create	☒ Alert ☒ Report
Action On Edit	Allow	Operations On Edit	☒ Alert ☐ Report
Alert Text	Use one of these records?		
Active	☒		
Matching Rule	☒ Weather_Matching_Rule ☒ Mapped		
Conditions	(Weather: City EQUALS Mumbai) AND (Weather: City EQUALS karnataka)		
Created By	chanda indraja, 9/25/2025, 3:10 AM	Modified By	chanda indraja, 9/25/2025, 3:14 AM

EditDeleteCloneDeactivate

4. Data Export & Backup

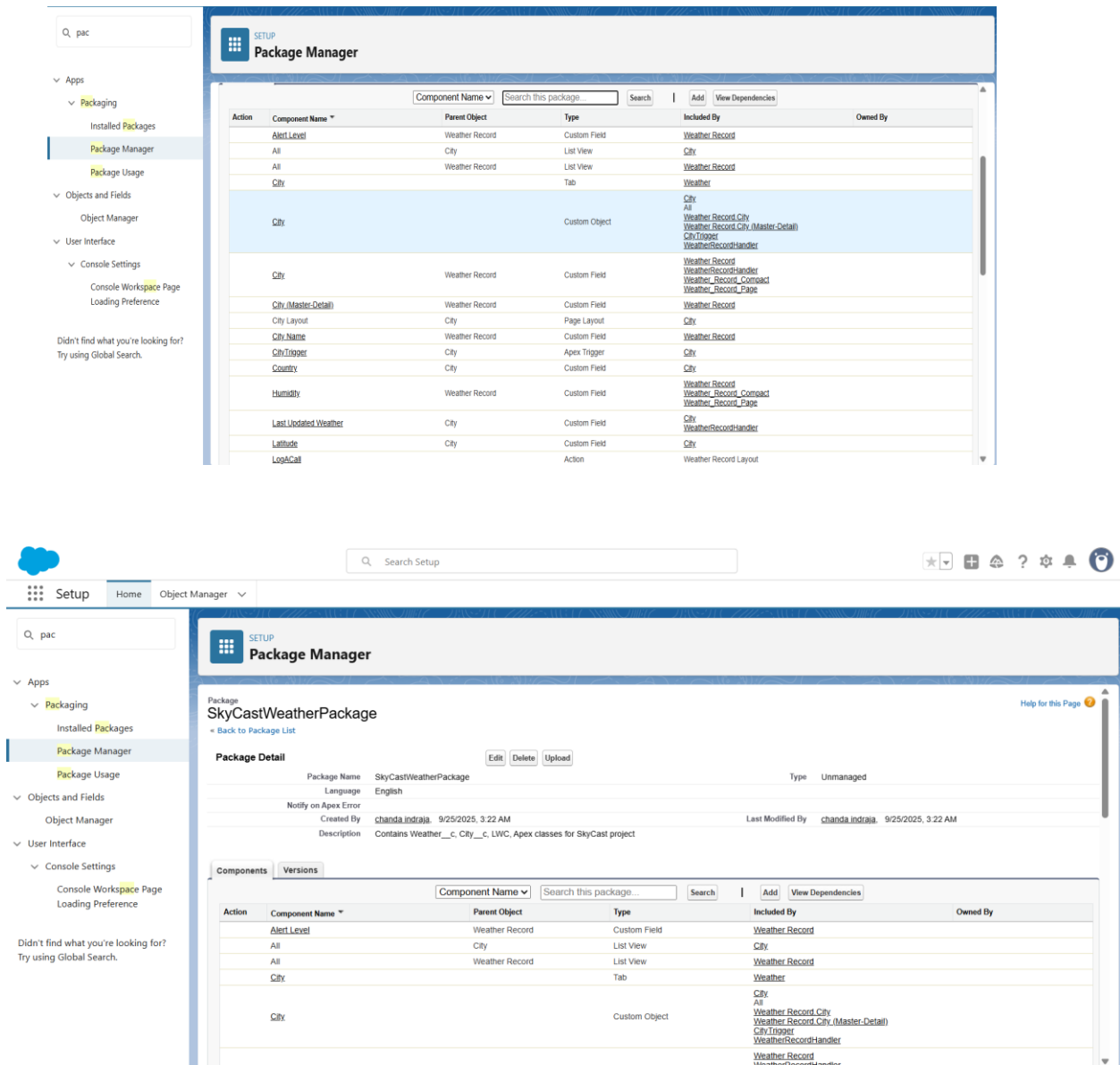
- **Purpose:** Protect against data loss by exporting Salesforce data.
- **Methods:**
 - **Data Export Wizard** → Manual or Scheduled CSV export of all objects.
 - **Data Loader Export** → Export selected objects or queries.
- **Features:**
 - Export can include **attachments, documents, images**.
 - Can schedule weekly or monthly backups.
- **Best Use Case:** Regular data backup for compliance & recovery.

5. Change Sets

- **Purpose:** Deploy metadata (configuration/customizations) between Salesforce orgs (Sandbox → Production).
- **Features:**
 - Supports objects, fields, workflows, flows, validation rules, etc.
 - Point-and-click interface (no coding).
 - Requires **connected sandbox/production orgs**.
- **Limitations:** Cannot deploy some metadata types (e.g., standard picklist values).
- **Best Use Case:** Admin-friendly deployments.

6. Unmanaged vs Managed Packages

- **Unmanaged Package:**
 - Source code is visible.
 - Used for sharing open-source or sample apps.
 - No upgrade support.
- **Managed Package:**
 - Code is hidden (IP protected).
 - Supports **version upgrades**.
 - Used by ISVs (Independent Software Vendors) for AppExchange apps.
- **Best Use Case:**
 - *Unmanaged:* Internal projects & learning.
 - *Managed:* Commercial apps on AppExchange.

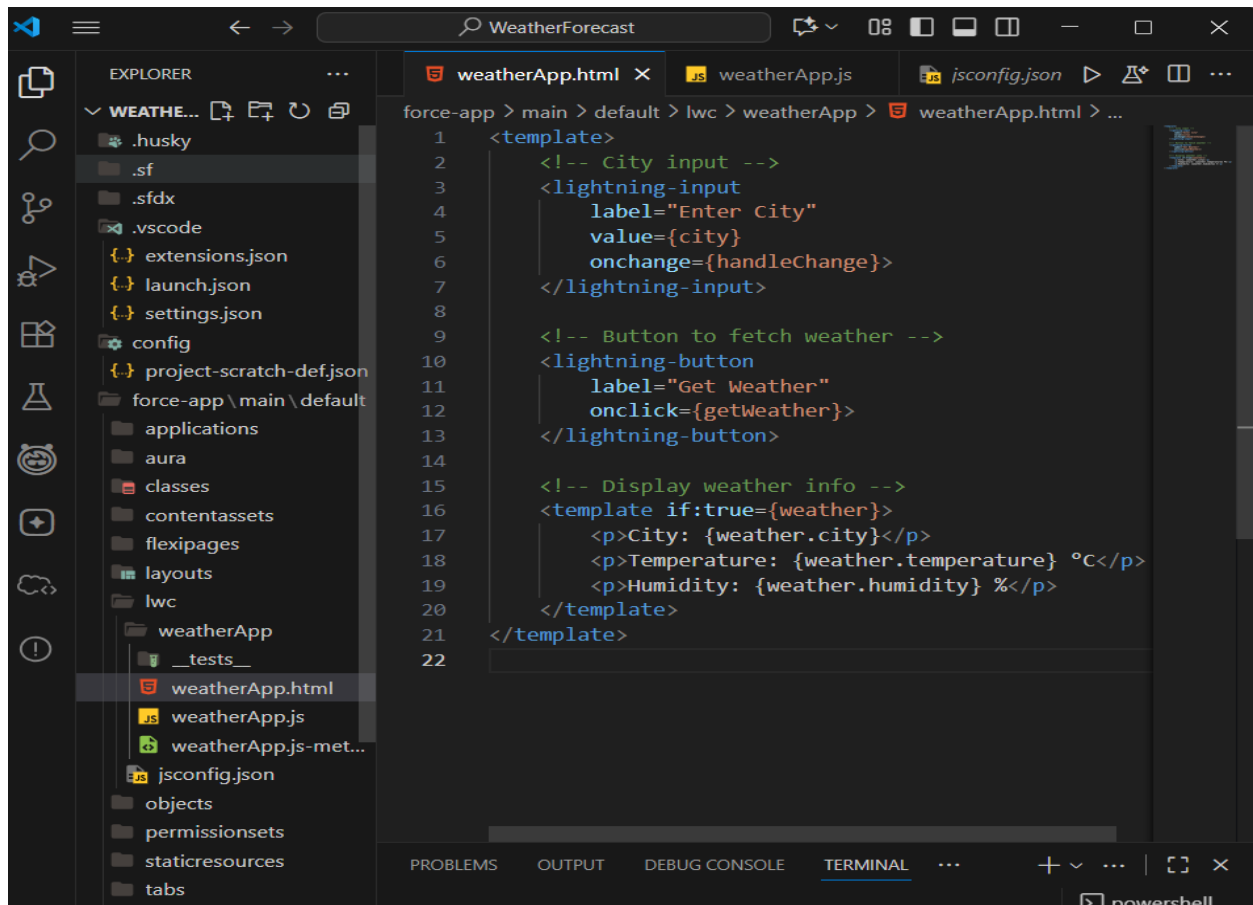


7. ANT Migration Tool

- **Purpose:** Command-line utility for automating metadata deployment.
- **Features:**
 - Based on **Apache ANT + ant-salesforce.jar**.
 - Supports scripted **retrieve & deploy**.
 - Uses `build.xml` + `package.xml` to define deployment.
- **Best Use Case:** Automated deployments in CI/CD pipelines.
- **Requirement:** Java & ANT installed locally.

8. VS Code & SFDX (Salesforce DX)

- **Purpose:** Modern development & deployment tool for Salesforce projects.
- **Features:**
 - Uses **Salesforce CLI (SFDX)** for source-driven development.
 - Supports Git-based version control.
 - Create & manage **scratch orgs** for testing.
 - Retrieve, deploy, and test metadata.
- **Best Use Case:** Developer-focused projects, CI/CD automation.
- **Setup:**
 1. Install **VS Code**.
 2. Install Salesforce **CLI (SFDX)**.
 3. Install Salesforce **Extension Pack** in VS Code.
 4. Authenticate org (`sfdx force:auth:web:login`).
 5. Retrieve/deploy metadata via commands.



Phase 9: Reporting, Dashboards & Security Review

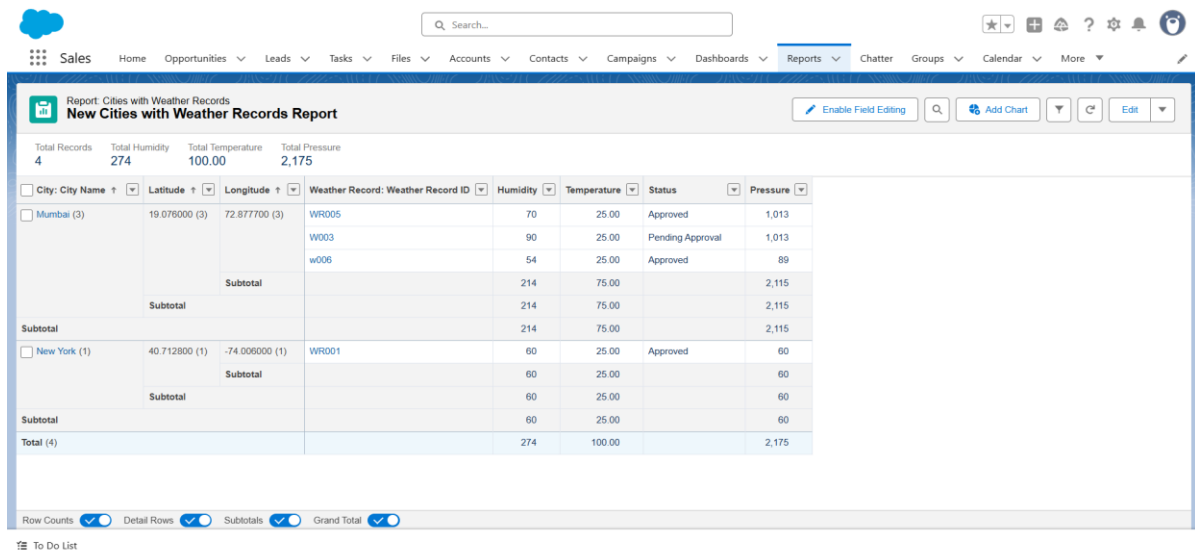
This phase focuses on analyzing business data through **Reports & Dashboards**, and ensuring **data security and compliance** using Salesforce's built-in security features.

1. Reports(Tabular, Summary, Matrix, Joined)

Reports help users analyze Salesforce data in different formats.

Types of Reports:

- **Tabular Reports**
 - Simple, spreadsheet-like format.
 - Best for **lists** (e.g., "All Accounts in New York").
 - Cannot create groupings or dashboards.
- **Summary Reports**
 - Similar to Tabular, but supports **grouping rows**.
 - Allows subtotals and charts.
 - Example: "Opportunities grouped by Stage."
- **Matrix Reports**
 - Group records by both **rows and columns**.
 - Best for comparing related totals.
 - Example: "Revenue by Sales Rep and by Quarter."
- **Joined Reports**
 - Combine multiple report blocks (each with different report type).
 - Best for complex reporting needs.
 - Example: "Cases with related Opportunities and Activities."



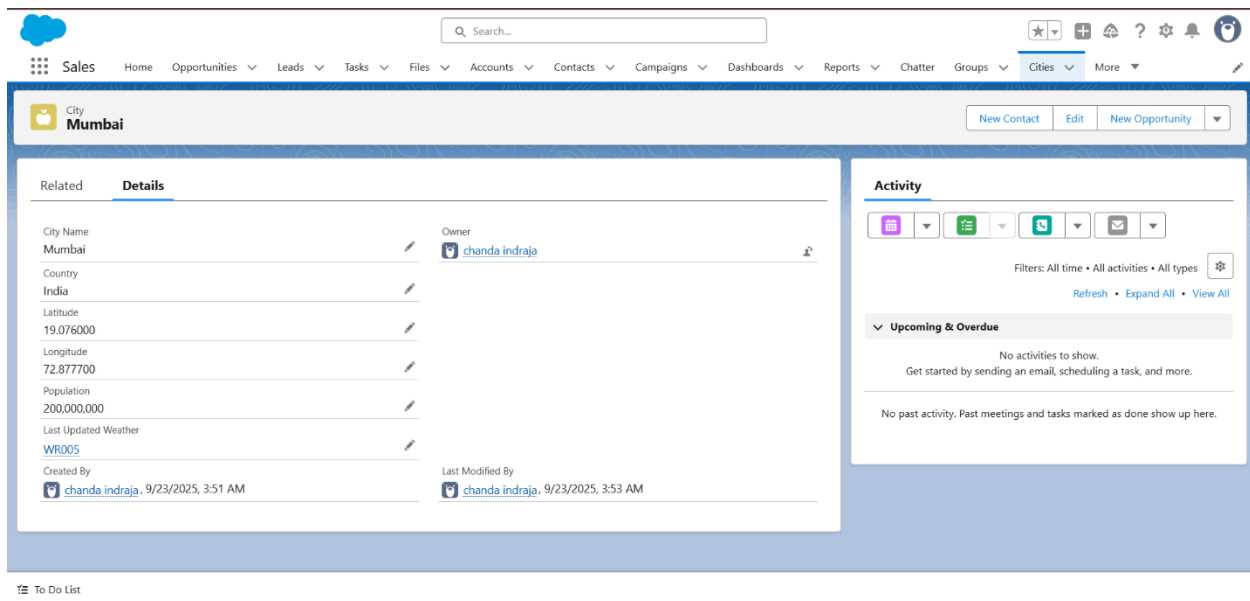
Report: Cities with Weather Records
New Cities with Weather Records Report

Enable Field Editing | Add Chart | Edit

City: City Name	Latitude	Longitude	Weather Record: Weather Record ID	Humidity	Temperature	Status	Pressure
Mumbai (3)	19.076000 (3)	72.877700 (3)	WR005	70	25.00	Approved	1,013
			W003	90	25.00	Pending Approval	1,013
			w006	54	25.00	Approved	89
		Subtotal		214	75.00		2,115
		Subtotal		214	75.00		2,115
		Subtotal		214	75.00		2,115
New York (1)	40.712800 (1)	-74.006000 (1)	WR001	60	25.00	Approved	60
		Subtotal		60	25.00		60
		Subtotal		60	25.00		60
		Subtotal		60	25.00		60
Total (4)				274	100.00		2,175

Row Counts: ☒ Detail Rows: ☒ Subtotals: ☒ Grand Total: ☒

To Do List



2. Report Types

Standard Report Types

- Auto-generated by Salesforce when you create **standard or custom objects**.
- Provide access to most fields and relationships of the object.
- Examples:
 - Accounts with Contacts
 - Opportunities with Products
- **Limitations:**
 - Cannot always include **custom relationships**.
 - Fields may be restricted compared to Custom Report Types.

Custom Report Types (CRTs)

- Created manually by Admin when **standard report types don't meet requirements**.
- Benefits:
 - Control which objects and fields are available in the report.
 - Define **object relationships** (e.g., "With" or "Without" records).
 - Add custom labels, descriptions, and categories for easier use.
- Example Use Case:
 - "Weather Records with related Cities and without Forecasts."

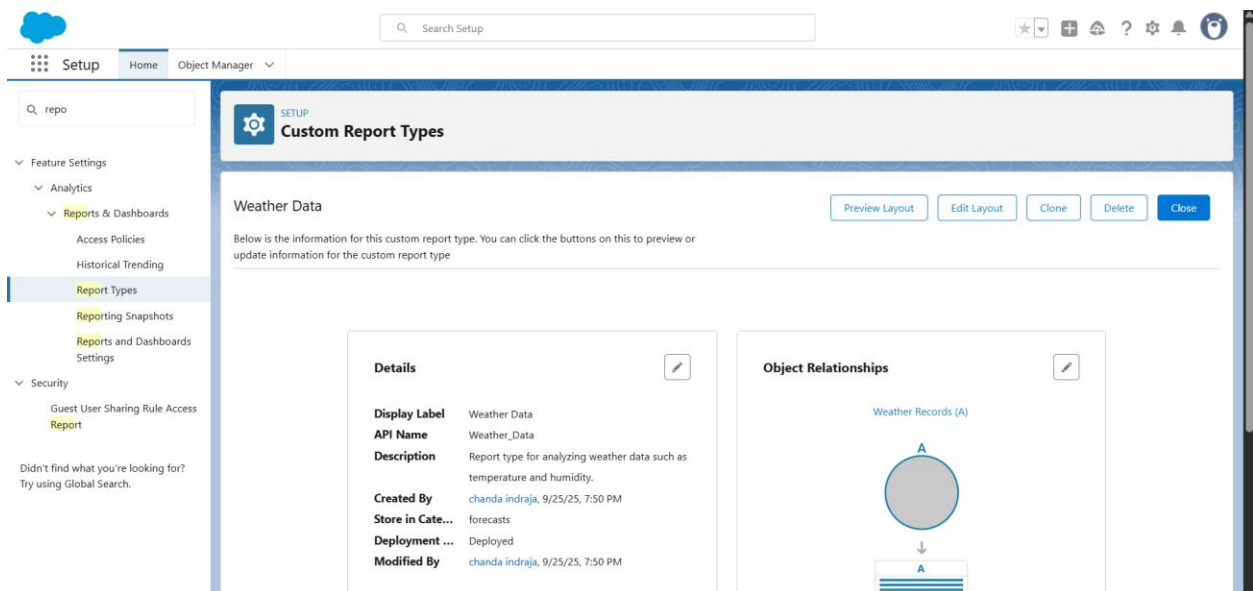
Primary Object

- The **main object** that the report focuses on.
- All records displayed will start from this object.
- Example:

- If **Weather__c** is the primary object → the report shows Weather data.

Related Objects

- Objects that are connected through **lookup or master-detail relationships**.
- Relationship types in CRTs:
 - **With** → Report includes related records.
 - **With or Without** → Includes both matching and non-matching records.
- Example:
 - Weather__c (Primary) → City__c (Related).

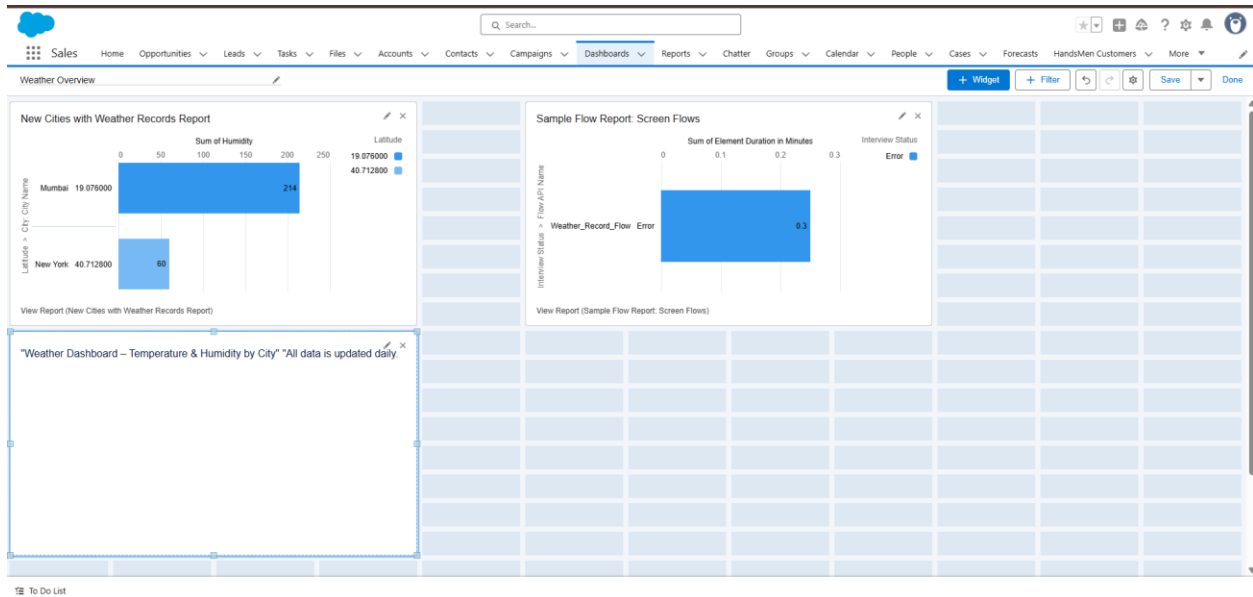


3. Dashboards

- **Purpose:** Visual representation of report data.
- **Components:** Charts, tables, metrics, gauges.
- **Source:** Each component is based on one underlying report.
- **Use Case:** "Executive Dashboard showing Sales Revenue, Pipeline, and Closed Deals."

Features:

- Interactive charts for quick insights.
- Supports multiple components on a single page.
- Can schedule dashboards to refresh automatically.

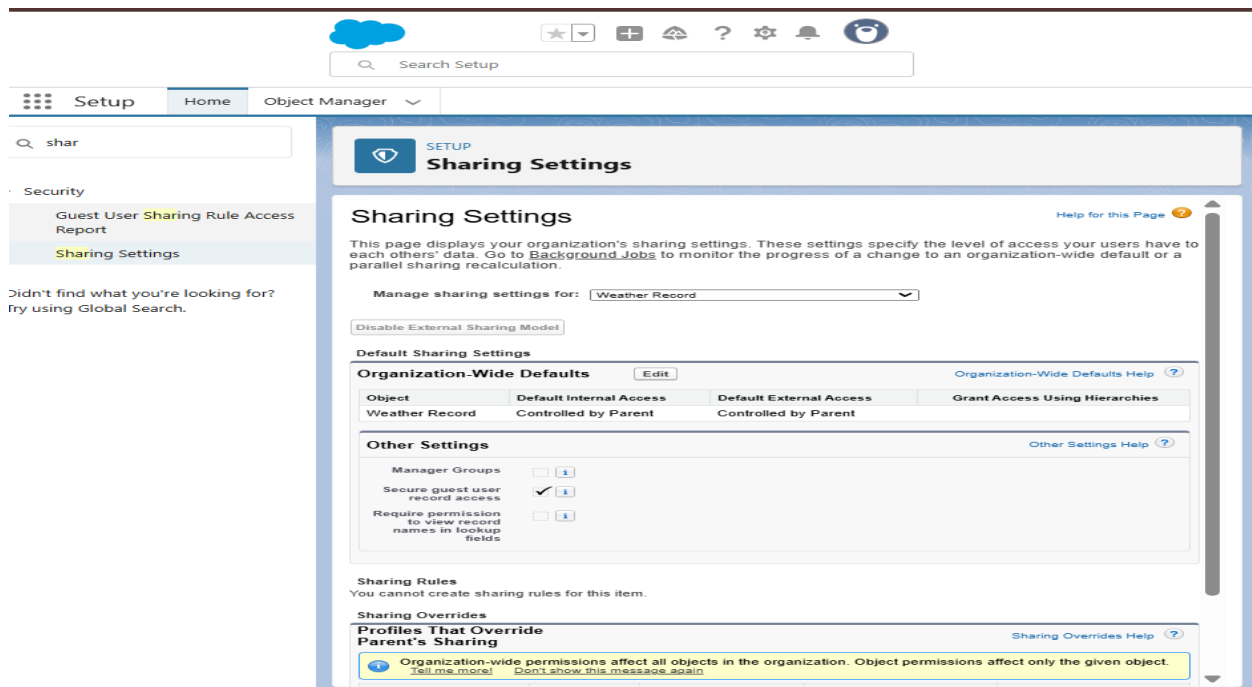


4. Dynamic Dashboards

- **Purpose:** Show data according to the **logged-in user's security and permissions**.
- **Benefits:**
 - No need to create multiple dashboards for different users.
 - Ensures users only see data they are authorized to view.
- **Best Use Case:** Sales Managers vs. Sales Reps viewing the same dashboard but with different data visibility.

5. Sharing Settings

- **Purpose:** Define how records are shared between users.
- **Options:**
 - **Private** → Only record owner & admins can access.
 - **Public Read Only** → Everyone can see but not edit.
 - **Public Read/Write** → Everyone can see & edit.
- **Tools:**
 - **Organization-Wide Defaults (OWD)** – baseline access.
 - **Role Hierarchy** – managers can access subordinates' data.
 - **Sharing Rules** – exceptions to open access.
 - **Manual Sharing** – record owner shares specific records.



6. Field-Level Security (FLS)

Purpose

- Control **who can see or edit** individual fields in Salesforce objects.
- Ensures **sensitive information** is protected while giving appropriate access to users.
- Works at the **Profile or Permission Set level**.

Options / Settings

- **Visible / Hidden**
 - Determines if a field is **shown or completely hidden** from users.
 - Hidden fields do not appear in **Page Layouts, Reports, or List Views**.
- **Read-Only**
 - Allows users to **view** the field but **cannot edit** it.
 - Useful for fields that should be **visible for reference** but protected from changes.
- **Editable**
 - Users can **view and update** the field.
 - Usually granted to trusted roles/profiles only.

7. Session Settings

Purpose

- Define **security policies for user sessions** in Salesforce.
- Protect sensitive data and prevent unauthorized access.

Key Features / Options

- **Session Timeout**
 - Auto-logout users after a period of inactivity.
 - Example: 15, 30, 60 minutes depending on org security requirements.
- **Restrict Concurrent Logins**
 - Prevent multiple logins using the same user credentials simultaneously.
- **IP Restrictions on Sessions**
 - Limit login or session access to trusted IP addresses.
- **Enable Multi-Factor Authentication (MFA)**
 - Adds an extra security layer using a verification code or app.

The screenshot shows the Salesforce Setup interface. The top navigation bar includes the Salesforce logo, a search bar labeled "Search Setup", and several utility icons. Below this, the "Setup" menu is open, showing "Home" and "Object Manager". The left sidebar contains a search bar with "sessi" and a list of security-related items: "Security", "Session Management", and "Session Settings". The main content area is titled "Session Settings" and includes a "Help for this Page" link. The page content is organized into sections: "Session Timeout" with a dropdown set to "2 hours" and checkboxes for "Disable session timeout warning popup" (unchecked) and "Force logout on session timeout" (checked); "Session Settings" with checkboxes for "Lock sessions to the IP address from which they originated" (unchecked), "Lock sessions to the domain in which they were first used" (checked), "Terminate all of a user's sessions when an admin resets that user's password" (unchecked), "Force relogin after Login-As-User" (checked), "Require HttpOnly attribute" (unchecked), "Use POST requests for cross-domain sessions" (unchecked), "Enforce login IP ranges on every request" (unchecked), and "When embedding a Lightning application in a third-party site, use a session token instead of a session cookie" (unchecked); and "Extended use of IE11 with Lightning Experience" at the bottom.

8. Login IP Ranges

Purpose

- Restrict access to Salesforce **based on trusted IP addresses**.
- Ensures that users can only log in from **approved networks**.

Setup

- Defined at the **Profile level** for each user type.
- Users outside the allowed IP range **cannot log in**.
- IP ranges can be set per profile, e.g., Sales team vs. Support team.

Best Use Cases

- Prevent unauthorized logins from **public networks**.
- Enforce **office network or VPN-only access**.
- Protect sensitive org data in multi-location environments.

The screenshot shows the Salesforce Setup interface. The left sidebar contains a search bar with "profiles" entered, and a list of items including "Users" and "Profiles" (highlighted). The main content area is titled "SETUP Profiles" and includes a "password emails" section with "Edit", "Clone", "Delete", and "View Users" buttons. Below this are several configuration sections: "Login Hours" (with an "Edit" button and "Login Hours Help" link), "Login IP Ranges" (with a "New" button and "Login IP Ranges Help" link), "Enabled Apex Class Access" (with an "Edit" button and "Enabled Apex Class Access Help" link), "Enabled Visualforce Page Access" (with an "Edit" button and "Enabled Visualforce Page Access Help" link), and "Enabled External Data Source Access" (with an "Edit" button and "Enabled External Data Source Access Help" link). The "Login IP Ranges" section contains a table with the following data:

Action	IP Start Address	IP End Address	Description
Edit Del	13.108.0.0	255.255.255.255	

9. Audit Trail

Purpose

- Track all **configuration changes** in Salesforce setup.
- Useful for **troubleshooting, compliance, and security audits**.

Key Features

- **Tracks Setup Changes**
 - Examples: fields, objects, workflows, validation rules, permissions.
- **History Availability**
 - Last 20 entries visible directly in Setup Audit Trail.
 - Full 180-day history downloadable as CSV for long-term auditing.
- **User Information Tracked**
 - Who made the change, what was changed, and when it occurred.

Best Practices

- Regularly **download audit trail data** for compliance reviews.
- Use in conjunction with **Field-Level Security and Sharing Rules** for complete visibility.
- Helps in **troubleshooting configuration issues** when errors occur.

Conclusion

The **SkyCast Weather Information System** provides a robust and scalable solution for managing, analyzing, and reporting weather-related data. By leveraging Salesforce features such as **custom objects, data import tools, reporting, dashboards, and security controls**, the system ensures accurate data capture, efficient processing, and secure access for authorized users. With integrated reporting and dashboards, stakeholders can monitor weather patterns in real-time, make informed decisions, and maintain compliance with organizational security standards. Overall, SkyCast delivers a comprehensive platform to transform raw weather data into actionable insights, enhancing operational efficiency and decision-making capabilities.

Phase 10: Final Presentation & Demo Day

1. Pitch Presentation

The pitch presentation is the **final opportunity to showcase the project idea, goals, and impact**.

- Provide a **clear problem statement** and how your solution addresses it.
- Highlight the **core features and technical stack** used in development.
- Emphasize the **business or user value** of your project.
- Keep it concise, engaging, and structured like a professional product pitch.

2. Demo Walkthrough

The demo walkthrough is a **live or recorded demonstration** of the project.

- Show the **end-to-end functionality** of your application.
- Highlight important modules, user flows, and unique features.
- Demonstrate real-world use cases to prove your project's usefulness.
- Avoid technical jargon for non-technical audiences; instead, focus on usability.

Demo Video:

 [Watch the demo video here:](https://drive.google.com/file/d/14B5Dnv9HnYJh7_NO56U8xlayokpPcb_/view?usp=drive_link)

https://drive.google.com/file/d/14B5Dnv9HnYJh7_NO56U8xlayokpPcb_/view?usp=drive_link

3. Feedback Collection

Feedback is crucial to understand how others **perceive your project and where it can improve**.

- Collect input from **mentors, peers, and potential users**.
- Use **structured feedback forms or surveys** for clear insights.
- Pay attention to both **technical improvements** and **user experience suggestions**.
- Record common feedback themes for future iterations.

4. Handoff Documentation

Handoff documentation ensures your project can be **maintained, extended, or deployed** by others.

- Provide **setup instructions** (installation, environment, dependencies).
- Include **user guides** (how to run and use the system).
- Maintain **technical documentation** (architecture diagrams, APIs, database schema).
- Store all docs in your GitHub repo for easy access.

Conclusion

The **SkyCast Weather Information System** provides a robust and scalable solution for managing, analyzing, and reporting weather-related data. By leveraging Salesforce features such as **custom objects, data import tools, reporting, dashboards, and security controls**, the system ensures accurate data capture, efficient processing, and secure access for authorized users. With integrated reporting and dashboards, stakeholders can monitor weather patterns in real-time, make informed decisions, and maintain compliance with organizational security standards. Overall, SkyCast delivers a comprehensive platform to transform raw weather data into actionable insights, enhancing operational efficiency and decision-making capabilities.